

Chapter 09

Custom Classes

Learning Objectives

- Define a custom class and use it to create objects.
- Implement a class's behaviors as member functions and attributes as data members.
- Enforce data encapsulation of private data members using public get and set functions.
- Use a constructor to initialize an object's data.
- Separate a class's interface from its implementation for reuse.
- Access class members via dot (.) and arrow (->) operators.
- Use destructors to perform "termination housekeeping" as objects go out of scope.
- Assign the data members of one object to those of another.
- Create objects composed of other objects.
- Use friend functions and declare friend classes.
- Access non-static class members via the this pointer.
- Use static data members and member functions.
- Create aggregate types via structs and use C++20 designated initializers to initialize aggregate members.

Introduction

- We have presented a friendly introduction to object orientation, discussing classes, objects, data members (attributes) and member functions (behaviors).
- In our Objects-Natural case studies, you've created objects of existing classes and called their member functions to perform powerful tasks without knowing how these classes worked internally.
- This chapter begins our deeper treatment of object-oriented programming as we craft **valuable custom classes**.
- C++ is an **extensible programming language**—each class you define becomes a new type you can use to create objects.
 - Some development teams in industry work on applications that contain hundreds, or even thousands, of custom classes.

Test-Driving an Account Object

- We begin our introduction to custom classes with three examples that create objects of an Account class representing a simple bank account.
 - First, let's look at the main program and output, so you can see an object of our initial Account class in action.
 - To help you prepare for the larger programs you'll encounter later in this book and in your career, we define the Account class and main in separate files—main in AccountTest.cpp and class Account in Account.h.

Test-Driving an Account Object (Cont.)

```
1 // AccountTest.cpp
2 // Creating and manipulating an Account object.
3 #include <format>
4 #include <iostream>
5 #include <string>
6 #include "Account.h"
7
8 int main() {
9     Account myAccount{}; // create Account object myAccount
10
11     // show that the initial value of myAccount's name is the empty string
12     std::cout << std::format("Initial account name: {}\n",
13         myAccount.getName());
14
15     // prompt for and read the name
16     std::cout << "Enter the account name: ";
17     std::string name{};
18     std::getline(std::cin, name); // read a line of text
19     myAccount.setName(name); // put name in the myAccount object
20
21     // display the name stored in object myAccount
22     std::cout << std::format("Updated account name: {}\n",
23         myAccount.getName());
24 }
```

Initial account name:
Enter the account name: Olga Novikova
Updated account name: Olga Novikova

Test-Driving an Account Object (Cont.)

- Instantiating an Object

- Typically, you cannot call a class's member functions until you create an object of that class.

- Line 9

`Account myAccount{}; // create Account object myAccount`
creates an **object** called `myAccount`. The variable's type is `Account`—the class we'll define in the header file.

Test-Driving an Account Object (Cont.)

- Headers and Source-Code Files

- When we declare int variables, the compiler knows what int is—a fundamental type built into C++.
- In line 9, however, the compiler does not know in advance what type Account is—it's a user-defined type.
- When packaged properly, new classes can be reused by other programmers.
- It's customary to place a reusable class definition in a file known as a **header** with a **.h filename extension**.
 - ◆ You include that header wherever you need to use the class, as you've been doing with C++ standard library and third-party library classes.

Test-Driving an Account Object (Cont.)

- Headers and Source-Code Files

- We tell the compiler what an Account is by including its header, as in line 6:

`#include "Account.h"`

- ◆ If we omit this, the compiler issues error messages wherever we use the class Account and its capabilities.
- A header that you define in your program is placed in double quotes ("") rather than the angle brackets (<>).
 - ◆ The double quotes tell the compiler to [check the folder containing AccountTest.cpp](#) before the compiler's header search path.

Test-Driving an Account Object.

- Calling Class Account's getName Member Function
 - Class Account's getName member function returns the name stored in a particular Account object.
 - Line 13 calls myAccount.getName() to get the myAccount object's initial name, which is the empty string.
- Calling Class Account's setName Member Function
 - The setName member function stores a name in a specific Account object.
 - Line 19 calls myAccount's setName member function to store name's value in myAccount.
- Displaying the Name That Was Entered by the User
 - To confirm that myAccount now contains the name you entered, line 23 calls myAccount's getName member function again and displays its result.

Account Class with a Data Member and Set and Get Member Functions

- Now that we've seen the Account class in action, we present its internal details.

```
1 // Account.h
2 // Account class with a data member and
3 // member functions to set and get its value.
4 #include <string>
5 #include <string_view>
6
7 class Account {
8 public:
9     // member function that sets m_name in the object
10    void setName(std::string_view name) {
11        m_name = name; // replace m_name's value with name
12    }
13
14    // member function that retrieves the account name from the object
15    const std::string& getName() const {
16        return m_name; // return m_name's value to this function's caller
17    }
18 private:
19     std::string m_name; // data member containing account holder's name
20 }; // end class Account
```

Account Class with a Data Member and Set and Get Member Functions

- Class Definition

- Class Account contains the data member `m_name` (line 19) to store the account holder's name.
 - ◆ Each object of a class has its own copy of the class's data members.

```
7  class Account {
8  public:
9      // member function that sets m_name in the object
10     void setName(std::string_view name) {
11         m_name = name; // replace m_name's value with name
12     }
13
14     // member function that retrieves the account name from the object
15     const std::string& getName() const {
16         return m_name; // return m_name's value to this function's caller
17     }
18 private:
19     std::string m_name; // data member containing account holder's name
20 }; // end class Account
```

Account Class with a Data Member and Set and Get Member Functions

- Class Definition

- Class Account also contains member functions:

- ◆ setName (lines 10–12) stores a name in an Account, and
 - ◆ getName (lines 15–17) retrieves a name from an Account.

```
7  class Account {  
8  public:  
9      // member function that sets m_name in the object  
10 void setName(std::string_view name) {  
11     m_name = name; // replace m_name's value with name  
12 }  
13  
14     // member function that retrieves the account name from the object  
15 const std::string& getName() const {  
16     return m_name; // return m_name's value to this function's caller  
17 }  
18 private:  
19     std::string m_name; // data member containing account holder's name  
20 }; // end class Account
```

Account Class with a Data Member and Set and Get Member Functions

- Keyword class and the Class Body
 - The class definition begins with the **keyword class** (line 7), followed immediately by the **class's name** (Account).
 - By convention:
 - ◆ each word in a class name starts with a **capital first letter** and
 - ◆ data-member and member-function names begin with a lowercase first letter, and each subsequent word begins with a capital first letter.
 - Every class's **body is enclosed in braces {}** (lines 7 and 20). The class definition **terminates with a required semicolon** (line 20).

```
7  class Account {
```

```
20 }; // end class Account
```

Account Class with a Data Member and Set and Get Member Functions

- Data Member `m_name` of Type `std::string`
 - Recall that an object has **attributes**, which are implemented as **data members**.
 - Each object maintains its own copy of these throughout its lifetime—that is, while the object exists in memory.
 - Usually, a class also contains **member functions** that you can use to manipulate particular objects' data members.

Account Class with a Data Member and Set and Get Member Functions

- Data Member `m_name` of Type `std::string`
 - Data members are declared inside a class definition but outside the class's member functions.
 - By convention, C++ programmers typically place a class's data members **last in the class's body**.
 - You can declare the data members anywhere in the class outside its member function definitions, but scattering the data members can lead to hard-to-read code.

Account Class with a Data Member and Set and Get Member Functions

- Data Member `m_name` of Type `std::string`

- Line 19

- `std::string m_name; // data member containing account holder's name`
declares a string data member called `m_name`.

- ◆ The "`m_`" prefix is a common naming convention to indicate that a variable represents a **data member**.
 - ◆ Each Account objects has its own `m_name`.
 - ◆ Because `m_name` is a data member, it can be manipulated by the class's member functions.
 - ◆ Recall that the default string value is the empty string (`""`), which is why line 13 in `main` did not display a name.

Account Class with a Data Member and Set and Get Member Functions

- Use `std::` with Standard Library Components in Headers
 - Throughout `Account.h`, we use `std::` when referring to `string_view` (line 10) and `string` (lines 15 and 19).
 - Headers **should NOT** contain global scope **using directives** or **using declarations**.
 - ◆ These would be `#included` in other source files, potentially causing naming conflicts.
 - ◆ Since Chapter 6, we've qualified every standard library class name, function name and object (e.g., `std::cout`) with `std::` as a good practice.

```
10 void setName(std::string_view name) {  
15 const std::string& getName() const {  
19 std::string m_name; // data member containing account holder's name
```

Account Class with a Data Member and Set and Get Member Functions

- setName Member Function

- Member function setName (lines 10–12) receives a string_view representing the Account's name and assigns the name argument to data member m_name.
- Recall that a string_view is a **read-only view** into a character sequence, such as a std::string or a C-string.
- Line 11 copies the name parameter's string into m_name.
 - ◆ The "m_" in m_name makes it easy to distinguish the **parameter name** from the **data member m_name**.

```
10 void setName(std::string_view name) {  
11     m_name = name; // replace m_name's value with name  
12 }
```

Account Class with a Data Member and Set and Get Member Functions

- getName Member Function

- Member function getName (lines 15–17) has no parameters and returns a particular Account object's m_name to the caller as a `const std::string&`.
- Declaring the `returned reference const` ensures that the caller **CANNOT** modify the object's data via that reference.

```
15  const std::string& getName() const {  
16      return m_name; // return m_name's value to this function's caller  
17  }
```

Account Class with a Data Member and Set and Get Member Functions

- **const** Member Functions

- Note the **const** to the right of `getName`'s parameter list (line 15).
 - ◆ When returning `m_name`, member function `getName` does not, and should not, modify the Account object on which it's called.
 - ◆ Declaring a member function `const` tells the compiler, "this function should not modify the object on which it's called—if it does, please issue a compilation error."
 - This can help you locate errors if you accidentally insert code in this member function that would modify the object.
 - It also tells the compiler that `getName` **may be called on a const Account object** or via a reference or pointer to a `const Account`.

```
15  const std::string& getName() const {  
16      return m_name; // return m_name's value to this function's caller  
17  }
```

Account Class with a Data Member and Set and Get Member Functions

- Access Specifiers `private` and `public`
 - The keyword `private` (line 18) is an **access specifier**.
 - Each access specifier is followed by a required colon (:).
 - Data member `m_name`'s declaration (line 19) appears after `private`: to indicate that `m_name` is **accessible only to class Account's member functions**.
 - ◆ This is known as information hiding (or hiding implementation details) and is a recommended practice of the C++ Core Guidelines and object-oriented programming in general.
 - ◆ Data member `m_name` is encapsulated (hidden) and can be used only in class Account's `setName` and `getName` member functions.
 - ◆ Most data-member declarations appear after the access specifier **`private`**.

```
18     private:  
19     std::string m_name; // data member containing account holder's name
```

Account Class with a Data Member and Set and Get Member Functions

- Access Specifiers **private** and **public**

- This class also contains the **public** access specifier:
public:
- Class members listed **after public**—and before the next access specifier if there is one—are “**available to the public.**”
 - ◆ They can be used anywhere an object of the class is in scope.
 - ◆ Making a class’s data members private facilitates debugging because problems with data manipulations are localized to the class’s member functions.
 - Later, we’ll say more about private’s benefits, and introduce the access specifier protected.

Account Class with a Data Member and Set and Get Member Functions.

- Default Access for Class Members

- By **default**, class members are **private** unless you specify otherwise.
- After you list an access specifier, every subsequent class member has that access level until you list a different access specifier.
- The public and private access specifiers may be repeated, but this can be **confusing**.
- We prefer to **group** all of the class's public members under one public access specifier and group all of the class's private members under one private access specifier.

Account Class: Custom Constructors

- As you saw, creating an Account object initializes its string data member `m_name` to the empty string by default.
 - But what if you want to provide a name when creating an Account object?
- Each class can define **constructors** that specify custom initialization for objects of that class.
 - A constructor is a special member function that must **have the same name as the class**.
 - Constructors **cannot return values**, so they do not specify a return type, not even void.
 - C++ **guarantees** a constructor call when you create an object of a class type, so this is the ideal point to **initialize an object's data members**.
 - Every time you created an object, the corresponding class's constructor was called to initialize the object.
 - ◆ As you'll soon see, classes may have **overloaded constructors**.

Account Class: Custom Constructors (Cont.)

- Like member functions, constructors can have **parameters**—the corresponding argument values help initialize the object's data members.
- For example, you can specify an Account object's name when the object is created, as you'll do in line 10 of next program:

```
Account account1{"Mia Gonzalez"};
```

- Here, the "Mia Gonzalez" is **passed to the Account class's constructor** to initialize the account1 object's data.
- The preceding statement assumes that class Account has a constructor that can receive a string argument.

Account Class: Custom Constructors (Cont.)

- Account Class Definition

- The next program defines the Account class **with a constructor** that receives a name parameter and uses it to initialize the data member `m_name` when an Account object is created.

```
6 class Account {
7     public:
8         // constructor initializes data member m_name with the parameter name
9         explicit Account(std::string_view name)
10            : m_name{name} { // member initializer
11            // empty body
12            }
13
14        // function to set the account name
15        void setName(std::string_view name) {
16            m_name = name;
17        }
18
19        // function to retrieve the account name
20        const std::string& getName() const {
21            return m_name;
22        }
23    private:
24        std::string m_name; // account name
25    }; // end class Account
```

Account Class: Custom Constructors (Cont.)

- Account Class's Custom Constructor Definition
 - Lines 9–12 define Account's constructor.
 - ◆ Usually, constructors are **public**, so any code with access to the class definition can create and initialize objects of that class.
 - ◆ Line 9 indicates that the constructor has a `string_view` parameter.
 - When creating a new Account object, you must pass a person's name to the constructor's `string_view` parameter.
 - The constructor then initializes the data member `m_name` with the parameter's contents.
 - ◆ Line 9 does **not specify a return type**, not even `void`, because constructors cannot return values.
 - ◆ Also, constructors **cannot be declared `const`**—initializing an object must modify it.

```
8 // constructor initializes data member m_name with the parameter name
9 explicit Account(std::string_view name)
10 : m_name{name} { // member initializer
11 // empty body
12 }
```

Account Class: Custom Constructors (Cont.)

- Account Class's Custom Constructor Definition

- The constructor's member-initializer list (line 10)

```
: m_name{name}
```

initializes the m_name data member.

- ◆ Member initializers appear **between** a constructor's parameter list and the left brace that begins the constructor's body.
- ◆ You separate the member initializer list from the parameter list with a colon (:).
- ◆ Each member initializer consists of a data member's variable name followed by braces containing its initial value.
- ◆ This member initializer **calls the std::string class's constructor that receives a string_view**.
- ◆ If a class has more than one data member, each member initializer is separated from the next by a comma.
- ◆ Member constructors execute in the order you declare the data members in the class.
 - For clarity, list the member initializers in the same order—some compilers will warn you if you do not.
- ◆ The member-initializer list executes before the constructor's body.

Account Class: Custom Constructors (Cont.)

- Account Class's Custom Constructor Definition
 - Though you can perform initialization with assignment statements in the constructor's body, the C++ Core Guidelines recommend using member initializers.
 - You'll see later that [member initializers can be more efficient](#).
 - Also, you'll see that certain data members must be initialized using the member-initializer syntax because you cannot assign values to them in the constructor's body.

Account Class: Custom Constructors (Cont.)

- explicit Keyword

- Declare a constructor **explicit** if it can be called **with one argument**—that is, it has one parameter or it has additional parameters with default arguments.
 - ◆ This **prevents** the compiler from using the constructor to perform **implicit type conversions**.
 - ◆ The keyword **explicit** means that Account's constructor **must be called explicitly**, as in
Account account1{"Mia Gonzalez"};
- For now, simply declare all single-parameter constructors **explicit**.
 - ◆ Later, you'll see that single-parameter constructors without **explicit** can be called implicitly to perform **type conversions**.
 - ◆ Such implicit constructor calls can lead to **subtle errors** and are generally discouraged.

```
8 // constructor initializes data member m_name with the parameter name
9 explicit Account(std::string_view name)
10 : m_name{name} { // member initializer
11 // empty body
12 }
```

Account Class: Custom Constructors (Cont.)

- Test Driving Account Class

account1 name is: Mia Gonzalez
account2 name is: Asahi Susuk

```
1 // AccountTest.cpp
2 // Using the Account constructor to initialize the m_name
3 // data member when each Account object is created.
4 #include <format>
5 #include <iostream>
6 #include "Account.h"
7
8 int main() {
9     // create two Account objects
10    Account account1{"Mia Gonzalez"};
11    Account account2{"Asahi Susuki"};
12
13    // display each Account's corresponding name
14    std::cout << std::format(
15        "account1 name is: {}\naccount2 name is: {}\n",
16        account1.getName(), account2.getName());
17 }
```

Account Class: Custom Constructors (Cont.)

- Initializing Account Objects When They're Created
 - The main program initializes two Account objects using the constructor.
 - ◆ Line 10 creates the Account object named account1:
Account account1{"Mia Gonzalez"};
 - This calls the Account constructor (lines 9–12 of the header), which uses the argument "Mia Gonzalez" to initialize the new object's m_name data member.
 - ◆ Line 11 of main repeats this process, passing the argument "Asahi Susuki" to initialize m_name for the object account2:
Account account2{"Asahi Susuki"};

```
9 // create two Account objects
10 Account account1{"Mia Gonzalez"};
11 Account account2{"Asahi Susuki"};
```

Account Class: Custom Constructors (Cont.)

- Initializing Account Objects When They're Created
 - To confirm the objects were initialized properly, lines 14–16 call the getName member function on each Account object to get its name.
 - ◆ The output shows that each Account has a different name, confirming that each object has its own copy of data member m_name.

```
13 // display each Account's corresponding name
14 std::cout << std::format(
15     "account1 name is: {}\naccount2 name is: {}\n",
16     account1.getName(), account2.getName());
```

```
account1 name is: Mia Gonzalez
account2 name is: Asahi Susuk
```

Account Class: Custom Constructors (Cont.)

- Default Constructor

- Recall that we created an Account object **with empty braces** to the right of the object's variable name:
Account myAccount{};
 - ◆ In this statement, C++ **implicitly calls Account's default constructor**.
 - ◆ The compiler generates a **default constructor with no parameters in any class that does not define a constructor**.
 - ◆ The default constructor does not initialize the class's fundamental-type data members but does **call the default constructor for each data member** that's an object of another class.
 - For example, though you do not see this in the first Account class's code, Account's default constructor calls class std::string's default constructor to initialize the data member m_name to the empty string ("").
- When you declare an object with empty braces, as in the preceding statement, data members that are not explicitly initialized **get value initialized**.
 - ◆ Without the braces, such data members contain undefined ("garbage") values.

Account Class: Custom Constructors (Cont.)

- There's No Default Constructor in a Class That Defines a Constructor
 - The compiler will not create a default constructor for a class if you provide a custom constructor.
 - In that case, you will **NOT** be able to create an Account object by calling the constructor with no arguments **unless the custom constructor you define has an empty parameter list or has default arguments for all its parameters.**
 - We'll show later that you can force the compiler to create the default constructor even if you've defined non-default constructors.
 - **Unless default initialization** of your class's data members **is acceptable**, you should initialize them in their declarations or **provide a custom constructor** that initializes them with meaningful values.

Account Class: Custom Constructors.

- C++'s Special Member Functions

- In addition to the default constructor, the compiler can generate **default versions of five other special member functions**—a copy constructor, a move constructor, a copy assignment operator, a move assignment operator and a destructor—as well as C++20's new three-way comparison operator.
- This chapter briefly introduces copy construction, copy assignment and destructors.
- Chapter 11 introduces the **three-way comparison operator** and discusses the details of all these special member functions, including
 - ◆ when you might need to define custom versions of each and
 - ◆ the various C++ Core Guidelines for these special member functions.
- You'll see that you should design your classes so the compiler can auto-generate the special member functions for you.
 - ◆ This is called the “**Rule of Zero**,” meaning that you **provide no special member functions** in the class's definition.

Software Engineering with Set and Get Member Functions

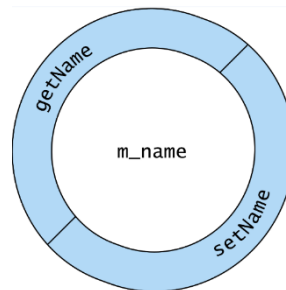
- As you'll see in the next section, **set member functions can validate** attempts to modify private data, and get member functions can control how that data is presented to the caller.
 - These are compelling software engineering benefits.
- A **client** of the class is any other code that **calls the class's member functions**.
 - If a data member were public, any client could see the data and do whatever it wanted with it, including setting it to an invalid value.
 - You might think that even though a client cannot directly access a private data member, the client can do whatever it wants with the variable by calling public set and get functions.
 - You'd think that you could call the public get function to peek at the private data and see exactly how it's stored in the object, and that you could call the public set function to modify the private data at will.

Software Engineering with Set and Get Member Functions

- Actually, **set** functions can be written to validate their arguments and **reject any attempts to set the data to incorrect values**, such as
 - a negative body temperature,
 - a day in March outside the range 1 through 31 or
 - a product code not in the company's product catalog.
- A **get** function can **present the data** in a different form, keeping the object's actual data representation **hidden from the user**.
 - For example, a Grade class might store a numeric grade as an int between 0 and 100, but a getGrade member function might return a letter grade as a string, such as "A" for grades between 90 and 100, "B" for grades between 80 and 89, etc.
 - Tightly controlling the access to and presentation of private data can **reduce errors** while increasing your programs' robustness, security and usability.

Software Engineering with Set and Get Member Functions.

- Conceptual View of an Account Object with Encapsulated Data
 - You can think of an Account object as shown in the following diagram.
 - ◆ The private data member `m_name` in the inner circle is hidden inside the object and accessible only via an outer layer of public member functions, represented by the shaded outer ring containing `getName` and `setName`.
 - ◆ Any client that needs to interact with the Account object can only call the public member functions of the outer layer.



Account Class with a Balance

- The next program defines an Account class that maintains two related pieces of data—the account holder's name and bank balance.
- The C++ Core Guidelines recommend defining related data items in a class (or in a struct).

```
1 // Account.h
2 // Account class with m_name and m_balance data members, and a
3 // constructor and deposit function that each perform validation.
4 #include <algorithm>
5 #include <string>
6 #include <string_view>
7
8 class Account {
9 public:
10     // Account constructor with two parameters
11     Account(std::string_view name, double balance)
12         : m_name{name}, m_balance{std::max(0.0, balance)} { // member init
13         // empty body
14     }
```

Account Class with a Balance (Cont.)

- Data Member balance

- A bank services many accounts, each with its own balance.
- Every Account object now has its own m_name and m_balance.
- Line 39 declares a double data member m_balance.

```
37 private:  
38     std::string m_name;  
39     double m_balance;  
40 }; // end class Account
```

Account Class with a Balance (Cont.)

- Two-Parameter Constructor

- The class has a constructor and four member functions.
- It's common for someone opening an account to deposit money immediately, so the constructor (lines 11–14) receives a second parameter balance (a double) representing the starting balance. We did **NOT** declare this constructor **explicit** because it cannot be called with only one parameter.

```
8  class Account {  
9      public:  
10         // Account constructor with two parameters  
11         Account(std::string_view name, double balance)  
12             : m_name{name}, m_balance{std::max(0.0, balance)} { // member init  
13             // empty body  
14     }
```

Account Class with a Balance (Cont.)

- Two-Parameter Constructor

- The `m_balance` member initializer calls the `std::max` function (header `<algorithm>`) to initialize `m_balance` to 0.0 or `balance`, whichever is greater.
 - ◆ This ensures `m_balance` has a **valid non-negative value** when creating an `Account` object. Later, we'll validate arguments in the constructor body and use exceptions to indicate invalid arguments.

```
8  class Account {
9  public:
10     // Account constructor with two parameters
11     Account(std::string_view name, double balance)
12     : m_name{name}, m_balance{std::max(0.0, balance)} { // member init
13         // empty body
14     }
```

Account Class with a Balance (Cont.)

- deposit Member Function

- Member function deposit (lines 17–21) receives a double parameter amount and does not return a value.
- Lines 18–20 ensure that parameter amount's value is added to m_balance only if amount is a valid deposit amount greater than zero.

```
16 // function that deposits (adds) only a valid amount to the balance
17 void deposit(double amount) {
18     if (amount > 0.0) { // if the amount is valid
19         m_balance += amount; // add it to m_balance
20     }
21 }
```

Account Class with a Balance (Cont.)

- getBalance Member Function

- Member function getBalance (lines 24–26) allows the class's clients to obtain an Account object's m_balance value.
- The member function specifies the return type double and an empty parameter list.
- It is **declared const** because returning m_balance's value does not, and should not, modify the Account object on which it's called.

```
23 // function that returns the account balance
24 double getBalance() const {
25     return m_balance;
26 }
```

Account Class with a Balance (Cont.)

- Manipulating Account Objects with Balances
 - The main function creates two Account objects (lines 8–9).
 - ◆ It initializes them with a valid balance of 50.00 and an invalid balance of -7.00, respectively.

```
1 // AccountTest.cpp
2 // Displaying and updating Account balances.
3 #include <format>
4 #include <iostream>
5 #include "Account.h"
6
7 int main() {
8     Account account1{"Mia Gonzalez", 50.00};
9     Account account2{"Asahi Susuki", -7.00};
```

Account Class with a Balance (Cont.)

- Manipulating Account Objects with Balances
 - Lines 12–15 output both Accounts' names and balances by calling each object's getName and getBalance member functions.
 - ◆ Our class ensures that initial balances are greater than or equal to zero.
 - ◆ The account2 object's balance is initially 0.0 because the constructor rejected the attempt to start account2 with a negative balance by setting its value to 0.0.

```
11 // display initial balance of each object
12 std::cout << std::format("account1: {} balance is ${:.2f}\n",
13     account1.getName(), account1.getBalance());
14 std::cout << std::format("account2: {} balance is ${:.2f}\n\n",
15     account2.getName(), account2.getBalance());
```

```
account1: Mia Gonzalez balance is $50.00
account2: Asahi Susuki balance is $0.00
```

Account Class with a Balance.

- Reading a Deposit Amount from the User and Making a Deposit
 - Lines 17–21 prompt for, input and display the account1 deposit amount.
 - Line 22 calls object account1's deposit member function, passing the amount to add to account1's balance.
 - Lines 25–28 output both Accounts' names and balances again to show that only account1's balance changed.

```
17     std::cout << "Enter deposit amount for account1: "; // prompt
18     double amount;
19     std::cin >> amount; // obtain user input
20     std::cout << std::format(
21         "adding ${:.2f} to account1 balance\n\n", amount);
22     account1.deposit(amount); // add to account1's balance
23
24     // display balances
25     std::cout << std::format("account1: {} balance is ${:.2f}\n",
26         account1.getName(), account1.getBalance());
27     std::cout << std::format("account2: {} balance is ${:.2f}\n\n",
28         account2.getName(), account2.getBalance());
```

```
Enter deposit amount for account1: 25.37
adding $25.37 to account1 balance
```

```
account1: Mia Gonzalez balance is $75.37
account2: Asahi Susuki balance is $0.00
```

Account Class with a Balance.

- Reading a Deposit Amount from the User and Making a Deposit
 - Lines 30–33 prompt for, input and display account2's deposit amount.
 - Line 34 calls object account2's deposit member function with the argument amount to add that value to account2's balance.
 - Finally, lines 37–40 output both Accounts' names and balances again to show that only account2's balance changed.

```
30     std::cout << "Enter deposit amount for account2: "; // prompt
31     std::cin >> amount; // obtain user input
32     std::cout << std::format(
33         "adding ${:.2f} to account2 balance\n\n", amount);
34     account2.deposit(amount); // add to account2 balance
35
36     // display balances
37     std::cout << std::format("account1: {} balance is ${:.2f}\n",
38         account1.getName(), account1.getBalance());
39     std::cout << std::format("account2: {} balance is ${:.2f}\n",
40         account2.getName(), account2.getBalance());
```

```
Enter deposit amount for account2: 123.45
adding $123.45 to account2 balance
```

```
account1: Mia Gonzalez balance is $75.37
account2: Asahi Susuki balance is $123.45
```

Time Class Case Study: Separating Interface from Implementation

- Each of our prior custom class definitions placed a class in a header for reuse, then included the header into a source-code file containing main, so we could create and manipulate objects of the class.
- Unfortunately, placing a complete class definition in a header **reveals the class's entire implementation** to its clients.
 - A header is simply a text file that anyone can open and read.
- Conventional software-engineering wisdom says that to use an object of a class, the client code (e.g., main) needs to know only
 - which member functions to call,
 - which arguments to provide to each member function and
 - which return type to expect from each member function.

Time Class Case Study: Separating Interface from Implementation

- The client code does not need to know how those functions are implemented.
 - This is another example of the principle of least privilege.
 - If the client-code programmer knows how a class is implemented, the programmer might write client code based on the class's implementation details.
 - If that implementation changes, the class's clients should not have to change.
 - Hiding the class's implementation details makes it easier to change the implementation while minimizing and hopefully eliminating changes to client code.

Time Class Case Study: Separating Interface from Implementation (Cont.)

- Our next example creates and manipulates an object of class Time.
- We demonstrate two important C++ software engineering concepts:
 - Separating interface from implementation, and
 - using the preprocessor directive “**#pragma once**” in a header to prevent the header code from being included in the same source-code file more than once—a class can be defined only once, so this prevents multiple-definition errors.

Time Class Case Study: Separating Interface from Implementation (Cont.)

- C++20 Modules Change How You Separate Interface from Implementation
 - As you'll see in Chapter 16, C++20 modules eliminate the need for preprocessor directives like `#pragma once`.
 - You'll also see that modules enable you to separate interface from implementation in a single source-code file or by using multiple source-code files.

Time Class Case Study: Separating Interface from Implementation (Cont.)

- Interface of a Class

- Interfaces define and standardize how people and systems interact with one another.
 - ◆ For example, a radio's controls serve as an interface between users and its internal components.
 - ◆ The controls allow users to perform specific operations, such as changing the station, adjusting the volume and choosing between AM and FM stations.
 - ◆ Various radios may implement these operations differently—some provide buttons, some provide dials and some support voice commands.
 - ◆ The **interface specifies what** operations a radio permits users to perform **but does not specify how** the operations are implemented inside the radio.

Time Class Case Study: Separating Interface from Implementation (Cont.)

- Interface of a Class

- Similarly, the interface of a class describes what services a class's clients can use and how to request those services, but not how the class implements them.
- A class's public interface consists of the class's public member functions (also known as the class's public services).
- As you'll soon see, you can specify a class's interface by writing a class definition that lists only the class's member-function prototypes in the class's public section.

Time Class Case Study: Separating Interface from Implementation (Cont.)

- Separating the Interface from the Implementation
 - To separate the class's interface from its implementation, we break up class Time into two files—Time.h defines class Time and Time.cpp defines class Time's member functions.
 - This split
 - ◆ helps make the class reusable,
 - ◆ ensures that the clients of the class know what member functions the class provides, how to call them and what return types to expect, and
 - ◆ enables the clients to ignore how the class's member functions are implemented.
 - In addition, this split can **reduce compilation time** because the implementation file can be compiled, then does not need to be recompiled unless the implementation changes.
 - By convention, member-function definitions are placed in a .cpp file with the same base name (e.g., Time) as the class's header.
 - Some compilers support other filename extensions as well.

Time Class Case Study: Separating Interface from Implementation (Cont.)

- Class Definition

- The header Time.h contains Time's class definition (lines 8–17).

```
1 // Time.h
2 // Time class definition.
3 // Member functions are defined in Time.cpp
4 #pragma once // prevent multiple inclusions of header
5 #include <string>
6
7 // Time class definition
8 class Time {
9     public:
10         void setTime(int hour, int minute, int second);
11         std::string to24HourString() const; // 24-hour string format
12         std::string to12HourString() const; // 12-hour string format
13     private:
14         int m_hour{0}; // 0 - 23 (24-hour clock format)
15         int m_minute{0}; // 0 - 59
16         int m_second{0}; // 0 - 59
17 };
```

Time Class Case Study: Separating Interface from Implementation (Cont.)

- Class Definition

- Rather than function definitions, the class contains function prototypes (lines 10–12) that describe the class's public interface without revealing the member-function implementations.
 - ◆ The function prototype in line 10 indicates that setTime requires three int parameters and returns void.
 - ◆ The prototypes for to24HourString and to12HourString (lines 11–12) specify that they take no arguments and return a string.
 - ◆ Classes with one or more constructors would also declare them in the header, as we'll do in subsequent examples.

```
9 public:
10     void setTime(int hour, int minute, int second);
11     std::string to24HourString() const; // 24-hour string format
12     std::string to12HourString() const; // 12-hour string format
```

Time Class Case Study: Separating Interface from Implementation (Cont.)

- Class Definition

- The header still specifies the class's private data members (lines 14–16).
 - ◆ Each uses an in-class initializer to set the data member to 0.
 - ◆ The compiler must know the class's data members to determine how much memory to reserve for each object of the class.
 - ◆ Including the header Time.h in the client code provides the compiler with the information it needs to ensure that the client code calls class Time's member functions correctly.

```
13 private:  
14     int m_hour{0}; // 0 - 23 (24-hour clock format)  
15     int m_minute{0}; // 0 - 59  
16     int m_second{0}; // 0 - 59  
17 };
```

Time Class Case Study: Separating Interface from Implementation (Cont.)

- Class Definition

- The C++ Core Guidelines recommend using **in-class initializers** when a data member should be initialized with a constant.
- The Core Guidelines also recommend that, when possible, you **initialize all your data members with in-class initializers and let the compiler generate a default constructor for your class**—this constructor can be more efficient than one you define.

Time Class Case Study: Separating Interface from Implementation (Cont.)

- **#pragma once**

- In larger programs, headers also will contain other definitions and declarations.
- Attempts to include a header multiple times (inadvertently) often occur in programs with many headers that might include other headers.
- This could lead to compilation errors if the same definition appears more than once in a preprocessed file.
- The `#pragma once` directive (line 4) prevents `time.h`'s contents from being included in the same source-code file more than once.
 - ◆ This is sometimes referred to as an **include guard**. In Chapter 16, we'll discuss how C++20 modules help prevent such problems.

```
4  #pragma once // prevent multiple inclusions of header
```

Time Class Case Study: Separating Interface from Implementation (Cont.)

- Member Functions

- Time.cpp defines class Time's member functions, which were declared in lines 10–12 of header.

```
1 // Time.cpp
2 // Time class member-function definitions.
3 #include <format>
4 #include <stdexcept> // for invalid_argument exception class
5 #include <string>
6 #include "Time.h" // include definition of class Time from Time.h
7
8 // set new Time value using 24-hour time
9 void Time::setTime(int hour, int minute, int second) {
10     // validate hour, minute and second
11     if ((hour < 0 || hour >= 24) || (minute < 0 || minute >= 60) ||
12         (second < 0 || second >= 60)) {
13         throw std::invalid_argument{
14             "hour, minute or second was out of range"};
15     }
16
17     m_hour = hour;
18     m_minute = minute;
19     m_second = second;
20 }
```

Time Class Case Study: Separating Interface from Implementation (Cont.)

- Member Functions

- For member functions `to24HourString` and `to12HourString`, the `const` keyword must appear in both the function prototypes (header file, lines 11–12) and the function definitions (source code file, lines 23 and 28).

```
11     std::string to24HourString() const; // 24-hour string format
12     std::string to12HourString() const; // 12-hour string format

22 // return Time as a string in 24-hour format (HH:MM:SS)
23 std::string Time::to24HourString() const {
24     return std::format("{:02d}:{:02d}:{:02d}", m_hour, m_minute, m_second);
25 }
26
27 // return Time as string in 12-hour format (HH:MM:SS AM or PM)
28 std::string Time::to12HourString() const {
29     return std::format("{:02d}:{:02d} {}",
30         ((m_hour % 12 == 0) ? 12 : m_hour % 12), m_minute, m_second,
31         (m_hour < 12 ? "AM" : "PM"));
32 }
```

Time Class Case Study: Separating Interface from Implementation (Cont.)

- Including the Class Header in the Source-Code File
 - To indicate that the member functions in Time.cpp are part of class Time, we must first include the Time.h header (source code file, line 6).
 - This allows us to use the class name Time in the Time.cpp file (lines 9, 23 and 28).
 - When compiling Time.cpp, the compiler uses the information in Time.h to ensure that
 - ◆ the first line of each member function matches its prototype in Time.h and
 - ◆ each member function knows about the class's data members and other member functions.

```
1 // Time.cpp
2 // Time class member-function definitions.
3 #include <format>
4 #include <stdexcept> // for invalid_argument exception class
5 #include <string>
6 #include "Time.h" // include definition of class Time from Time.h
```

Time Class Case Study: Separating Interface from Implementation (Cont.)

- Scope Resolution Operator (::)

- Each member function's name (lines 9, 23 and 28) is preceded by the class name and the scope resolution operator (::).
 - ◆ This “ties” them to the (now separate) Time class definition, which declares the class's members.
 - ◆ The `Time::` tells the compiler that each `member function is in that class's scope`, and its name is known to other class members.
- Without “`Time::`” preceding each function name, the compiler would treat these as global functions with no relationship to class Time.
 - ◆ Such functions, also called “free” functions, **cannot access Time's private data** or call the class's member functions without receiving a Time object or a pointer or reference to one.
 - ◆ So, the compiler would not be able to compile these functions because it would not know that class Time declares variables `m_hour`, `m_minute` and `m_second`.
 - ◆ This would cause **compilation errors** because `m_hour`, `m_minute` and `m_second` are not declared as local variables in each function, nor are they declared as global variables.

```
9  void Time::setTime(int hour, int minute, int second) {
23  std::string Time::to24HourString() const {
28  std::string Time::to12HourString() const {
```

Time Class Case Study: Separating Interface from Implementation (Cont.)

- Member Function setTime and Throwing Exceptions
 - Function setTime (lines 9–20) is a public function that declares three int parameters and uses them to set the time.
 - Lines 11–12 test each argument to determine whether the value is in range.

```
8 // set new Time value using 24-hour time
9 void Time::setTime(int hour, int minute, int second) {
10     // validate hour, minute and second
11     if ((hour < 0 || hour >= 24) || (minute < 0 || minute >= 60) ||
12         (second < 0 || second >= 60)) {
13         throw std::invalid_argument{
14             "hour, minute or second was out of range"};
15     }
16
17     m_hour = hour;
18     m_minute = minute;
19     m_second = second;
20 }
```

Time Class Case Study: Separating Interface from Implementation (Cont.)

- Member Function setTime and Throwing Exceptions
 - If so, lines 17–19 assign the values to the m_hour, m_minute and m_second data members, respectively.
 - ◆ The hour argument must be greater than or equal to 0 and less than 24 because the 24-hour time format represents hours as integers from 0 to 23.
 - ◆ Similarly, the minute and second arguments must be greater than or equal to 0 and less than 60.

```
8 // set new Time value using 24-hour time
9 void Time::setTime(int hour, int minute, int second) {
10     // validate hour, minute and second
11     if ((hour < 0 || hour >= 24) || (minute < 0 || minute >= 60) ||
12         (second < 0 || second >= 60)) {
13         throw std::invalid_argument{
14             "hour, minute or second was out of range"};
15     }
16
17     m_hour = hour;
18     m_minute = minute;
19     m_second = second;
20 }
```

Time Class Case Study: Separating Interface from Implementation (Cont.)

- Member Function setTime and Throwing Exceptions
 - If any of the values is outside its range, setTime throws an exception (lines 13–14) of type `invalid_argument` (header `<stdexcept>`), notifying the client code that an invalid argument was received.
 - As you saw previously, you can use `try...catch` to catch exceptions and attempt to recover from them, which we'll do in the main function.
 - ◆ The `throw` statement creates a new `invalid_argument` object, initializing it with a custom error message string.
 - ◆ After the exception object is created, the `throw` statement terminates function `setTime`.
 - ◆ Then, the exception is returned to the code that called `setTime`.

```
13     throw std::invalid_argument{  
14         "hour, minute or second was out of range"};
```

Time Class Case Study: Separating Interface from Implementation (Cont.)

- Member Function setTime and Throwing Exceptions
 - Invalid values cannot be stored in a Time object because
 - ◆ when a Time object is created, its default constructor is called, and each data member is initialized to 0, as specified in lines 14–16 of header—this is the equivalent of 12 AM (midnight)—and
 - ◆ all subsequent attempts by a client to modify the data members are scrutinized by function setTime.

```
13 private:  
14     int m_hour{0}; // 0 - 23 (24-hour clock format)  
15     int m_minute{0}; // 0 - 59  
16     int m_second{0}; // 0 - 59
```

Time Class Case Study: Separating Interface from Implementation (Cont.)

- Member Functions `to24HourString` and `to12HourString`
 - Member function `to24HourString` (lines 23–25) takes no arguments and returns a formatted 24-hour string with three colon-separated digit pairs.
 - ◆ So, if the time is 1:30:07 PM, the function returns "13:30:07".
 - ◆ Each `{:02d}` placeholder in line 24 formats an integer (`d`) in a field width of two.
 - ◆ The 0 before the field width indicates that values with fewer than two digits should be formatted with leading zeros.

```
23 std::string Time::to24HourString() const {  
24     return std::format("{:02d}:{:02d}:{:02d}", m_hour, m_minute, m_second);  
25 }
```

Time Class Case Study: Separating Interface from Implementation (Cont.)

- Member Functions to24HourString and to12HourString
 - Function to12HourString (lines 28–32) takes no arguments and returns a formatted 12-hour time string containing the m_hour, m_minute and m_second values separated by colons and followed by an AM or PM indicator (e.g., 10:54:27 AM and 1:27:06 PM).
 - ◆ The function uses the placeholder `{:02d}` to format m_minute and m_second as two-digit values with leading zeros, if necessary.

```
27 // return Time as string in 12-hour format (HH:MM:SS AM or PM)
28 std::string Time::to12HourString() const {
29     return std::format("{}:{:02d}:{:02d} {}",
30         ((m_hour % 12 == 0) ? 12 : m_hour % 12), m_minute, m_second,
31         (m_hour < 12 ? "AM" : "PM"));
32 }
```

Time Class Case Study: Separating Interface from Implementation (Cont.)

- Member Functions `to24HourString` and `to12HourString`
 - Line 30 uses the conditional operator (`?:`) to determine how `m_hour` should be formatted.
 - ◆ If `m_hour` is 0 or 12 (AM or PM, respectively), it appears as 12; otherwise, we use the remainder operator (`%`) to get a value from 1 to 11.
 - The conditional operator in line 31 determines whether to include AM or PM.

```
27 // return Time as string in 12-hour format (HH:MM:SS AM or PM)
28 std::string Time::to12HourString() const {
29     return std::format("{}:{:02d}:{:02d} {}",
30         ((m_hour % 12 == 0) ? 12 : m_hour % 12), m_minute, m_second,
31         (m_hour < 12 ? "AM" : "PM"));
32 }
```

Time Class Case Study: Separating Interface from Implementation (Cont.)

- Implicitly Inlining Member Functions

- If a member function is fully defined in a class's body (as in our Account class examples), it's **implicitly declared inline**.
 - ◆ This can improve performance.
 - ◆ Remember that the compiler reserves the right not to inline any function.
 - ◆ Similarly, optimizing compilers also reserve the right to inline functions even if they are not declared with the inline keyword, provided that the compiler has access to the function's definition.
- Only the simplest, most stable member functions (i.e., whose implementations are unlikely to change) and those that are the most performance-sensitive should be defined in the class header.
 - ◆ Every change to the header requires you to recompile every source-code file dependent on that header—a time-consuming task in large systems.

Time Class Case Study: Separating Interface from Implementation (Cont.)

- Member Functions vs. Global Functions
 - Member functions `to24HourString` and `to12HourString` take no arguments.
 - They implicitly know about and can access the data members for the `Time` object on which they're invoked.
 - This is a benefit of object-oriented programming. In general, member-function calls receive either no arguments or fewer arguments than function calls in nonobject-oriented programs.
 - This reduces the likelihood of passing wrong arguments, the wrong number of arguments or arguments in the wrong order.

Time Class Case Study: Separating Interface from Implementation (Cont.)

- Using Class Time

- Once class Time is defined, it can be used as a type in declarations, as in:

```
Time sunset{}; // object of type Time
```

```
std::array<Time, 5> arrayOfTimes{}; // std::array of 5 Time objects
```

```
Time& dinnerTimeRef{sunset}; // reference to a Time object
```

```
Time* timePtr{&sunset}; // pointer to a Time object \
```

Time Class Case Study: Separating Interface from Implementation (Cont.)

- Using Class Time

- The main.cpp creates and manipulates a Time object.
 - ◆ Separating Time's interface from the implementation of its member functions does not affect how this client code uses the class.
 - ◆ Line 8 includes Time.h so the compiler knows how much space to reserve for the Time object t (line 17) and can ensure that Time objects are created and manipulated correctly in the client code.

```
1 // main.cpp
2 // Program to test class Time.
3 // NOTE: This file must be linked with Time.cpp.
4 #include <format>
5 #include <iostream>
6 #include <stdexcept> // invalid_argument exception class
7 #include <string_view>
8 #include "Time.h" // definition of class Time from Time.h
```

Time Class Case Study: Separating Interface from Implementation (Cont.)

- Using Class Time

- Throughout the program, we display the Time object's string representations using the displayTime function (lines 11–14), which calls Time's to24HourString and to12HourString member functions.

```
10 // displays a Time in 24-hour and 12-hour formats
11 void displayTime(std::string_view message, const Time& time) {
12     std::cout << std::format("{}\n24-hour time: {}\n12-hour time: {}\n\n",
13                             message, time.to24HourString(), time.to12HourString());
14 }
```

Time Class Case Study: Separating Interface from Implementation (Cont.)

- Using Class Time

- Line 17 creates the Time object t. Recall that class Time does not define a constructor, so this statement calls the compiler-generated default constructor.
 - ◆ Thus, t's m_hour, m_minute and m_second are set to 0 via their initializers in class Time's definition.
- Then, line 19 displays the time in 24-hour and 12-hour formats to confirm that the members were correctly initialized.
- Line 20 sets a new valid time by calling member function setTime, and line 21 again shows the time in both formats.

```
16  int main() {  
17      Time t{}; // instantiate object t of class Time  
18  
19      displayTime("Initial time:", t); // display t's initial value  
20      t.setTime(13, 27, 6); // change time  
21      displayTime("After setTime:", t); // display t's new value
```

Time Class Case Study: Separating Interface from Implementation (Cont.)

- Calling setTime with Invalid Values

- To show that setTime validates its arguments, line 25 calls setTime with invalid arguments of 99 for the hour, minute and second parameters.
- We placed this statement in a **try block** (lines 24–26) in case setTime throws an **invalid_argument exception**, which it will do in this example.

```
23 // attempt to set the time with invalid values
24 try {
25     t.setTime(99, 99, 99); // all values out of range
26 }
27 catch (const std::invalid_argument& e) {
28     std::cout << std::format("Exception: {}\n\n", e.what());
29 }
30
31 // display t's value after attempting to set an invalid time
32 displayTime("After attempting to set an invalid time:", t);
```

Time Class Case Study: Separating Interface from Implementation (Cont.)

- Calling setTime with Invalid Values

- When the exception occurs, it's **caught** at lines 27–29, and line 28 displays the exception's error message by calling its what member function.
- Line 32 shows the time to confirm that setTime **did not change** the time when invalid arguments were supplied.

```
23 // attempt to set the time with invalid values
24 try {
25     t.setTime(99, 99, 99); // all values out of range
26 }
27 catch (const std::invalid_argument& e) {
28     std::cout << std::format("Exception: {}\n\n", e.what());
29 }
30
31 // display t's value after attempting to set an invalid time
32 displayTime("After attempting to set an invalid time:". t);
```

Time Class Case Study: Separating Interface from Implementation.

- Object Size

- People new to object-oriented programming often suppose that objects must be quite large because they contain data members and member functions.
 - ◆ Logically, this is true. You may think of objects as containing data and functions physically (and our discussion has certainly encouraged this view).
 - ◆ However, this is not the case.
- An object in memory **contains only data, not the class's member functions.**
 - ◆ The member functions' code is maintained separately from all objects of the class.
 - ◆ Each object needs its own data because the data usually varies among the objects.
 - ◆ The function code is the same for all objects of the class and can be shared among them.

Compilation and Linking Process

- Often a class's interface and implementation will be created by one programmer and used by a separate programmer who implements the client code.
- A class-implementation programmer responsible for creating a reusable Time class creates the header Time.h and the source-code file Time.cpp that #includes the header, then provides these files to the client- code programmer.
- A reusable class's source code often is available to client-code programmers as a library they can download from a website like github.com.
- The client-code programmer needs to know only Time's interface to use the class and must be able to compile Time.cpp and link its object code.
- Since the class's interface is part of the class definition in the Time.h header, the client-code programmer must #include this file in the client's source-code file.
- The compiler uses the class definition in Time.h to ensure that the client code correctly creates and manipulates Time objects.

Compilation and Linking Process (Cont.)

- The last step to create the executable Time application is to [link](#)
 - the object code for the main function (that is, the client code),
 - the object code for class Time's member-function implementations and
 - the C++ standard library object code for the C++ classes (such as `std::string`) used by the class-implementation programmer and the client-code programmer.
- The linker's output for the previous program is the executable application that users can run to create and manipulate a Time object.
- Compilers and IDEs typically invoke the linker for you after compiling your code.

Compilation and Linking Process.

- Compiling Programs Containing Two or More Source-Code Files
 - We have showed how to compile and run C++ applications that contained one sourcecode (.cpp) file.
 - To compile and link multiple source-code files:
 - ◆ In Microsoft Visual Studio,
 - add to your project all the custom headers and source-code files that make up the program, then build and run the project.
 - You can place the headers in the project's Header Files folder and the source-code files in the project's Source Files folder, but these are mainly for organizing files in large projects.
 - The programs will compile if you place all the files in the Source Files folder.
 - ◆ For g++ or clang++
 - at the command line, open a shell and change to the directory containing the program's files.
 - Then in your compilation command, either list each .cpp file by name or use *.cpp to compile all the .cpp files in the current folder.
 - The preprocessor automatically locates the program-specific headers in that folder.

Class Scope and Accessing Class Members

- A class's data members and member functions belong to that class's scope.
- Non-member functions are defined at global namespace scope by default. (We discuss namespaces in more detail in Chapter 20.)
- Within a class's scope, members are immediately accessible by all of that class's member functions and can be referenced by name.

Class Scope and Accessing Class Members (Cont.)

- Outside a class's scope, public class members are referenced through
 - an object name,
 - a reference to an object,
 - a pointer to an object or
 - a pointer to a specific class member (discussed briefly in Chapter 20).
- We refer to these as handles on an object.
 - The handle's type helps the compiler determine the interface (that is, the member functions) accessible to the client via that handle.
 - We'll see soon later that the compiler inserts an implicit handle called the **this pointer** each time you refer to a data member or member function from within an object.

Class Scope and Accessing Class Members (Cont.)

- Dot (.) and Arrow (->) Member-Selection Operators
 - As you know, you can use an object's name or a reference to an object followed by the dot member-selection operator (.) to access the object's members.
 - To reference an object's members via a pointer to an object, follow the pointer name by the arrow member-selection operator (->) and the member name, as in `pointerName->memberName`.

Class Scope and Accessing Class Members.

- Accessing public Class Members Through Objects, References and Pointers
 - Consider an Account class that has a public deposit member function. Given the following declarations:
`Account account{}; // an Account object`
`Account& ref{account}; // ref refers to an Account object`
`Account* ptr{&account}; // ptr points to an Account object`
 - you can invoke member function deposit using the dot (.) and arrow (->) member selection operators as follows:
`account.deposit(123.45); // call deposit via account object's name`
`ref.deposit(123.45); // call deposit via reference to account object`
`ptr->deposit(123.45); // call deposit via pointer to account object`
 - Again, you should use references in preference to pointers whenever possible. We'll continue to show pointers when required and to prepare you to work with them in the legacy code you'll encounter in industry.

Access Functions and Utility Functions

- Access Functions

- Access functions can read or display data but not modify it. Another use of access functions is to test whether a condition is true or false.
- Such functions are often called **predicate functions**.
 - ◆ An example would be a `std::array`'s or a `std::vector`'s `empty` function.
 - ◆ A program might test `empty` before attempting to read an item from the container object.

- Utility Functions

- A utility function (also called a helper function) is a **private member function that supports the operation** of a class's other member functions and is not intended for use by the class's clients.
- Typically, a utility function contains code that would otherwise be duplicated in several other member functions.
- Many codebases have naming conventions for private member functions, such as preceding their names with an underscore (`_`).

Time Class Case Study: Constructors with Default Arguments

- The next program enhances class Time to demonstrate a constructor with default arguments.

```
1 // Time.h
2 // Time class containing a constructor with default arguments.
3 // Member functions defined in Time.cpp.
4 #pragma once // prevent multiple inclusions of header
5 #include <string>
6
7 // Time class definition
8 class Time {
9 public:
10     // default constructor because it can be called with no arguments
11     explicit Time(int hour = 0, int minute = 0, int second = 0);
12
13     // set functions
14     void setTime(int hour, int minute, int second);
15     void setHour(int hour); // set hour (after validation)
16     void setMinute(int minute); // set minute (after validation)
17     void setSecond(int second); // set second (after validation)
18
19     // get functions
20     int getHour() const; // return hour
21     int getMinute() const; // return minute
22     int getSecond() const; // return second
23
24     std::string to24HourString() const; // 24-hour time format string
25     std::string to12HourString() const; // 12-hour time format string
26 private:
27     int m_hour{0}; // 0 - 23 (24-hour clock format)
28     int m_minute{0}; // 0 - 59
29     int m_second{0}; // 0 - 59
30 };
```

Time Class Case Study: Constructors with Default Arguments (Cont.)

- Like other functions, constructors can specify **default arguments**.
- Line 11 declares a Time constructor with the default argument value 0 for each parameter.
 - A constructor with default arguments for all its parameters is also a **default constructor**—that is, it can be invoked with no arguments.
 - There can be **only one** default constructor per class.
 - Any change to a function's default argument values requires the client code to be recompiled (to ensure that the program still functions correctly).

```
10 // default constructor because it can be called with no arguments
11 explicit Time(int hour = 0, int minute = 0, int second = 0);
```

Time Class Case Study: Constructors with Default Arguments (Cont.)

- Class's Constructor

- Lines 9–11 define the Time constructor, which calls setTime to validate and assign values to the data members.

```
8 // Time constructor initializes each data member
9 Time::Time(int hour, int minute, int second) {
10     setTime(hour, minute, second);
11 }
```

Time Class Case Study: Constructors with Default Arguments (Cont.)

- Class's Constructor

- Function setTime (lines 14–31) ensures that hour is in the range 0–23 and minute and second are each in the range 0–59.
 - ◆ If any argument is out of range, setTime throws an exception—in which case, the Time object will not complete construction and will not exist for use in the program.

```
13 // set new Time value using 24-hour time
14 void Time::setTime(int hour, int minute, int second) {
15     // validate hour, minute and second
16     if (hour < 0 || hour >= 24) {
17         throw std::invalid_argument{"hour was out of range"};
18     }
19
20     if (minute < 0 || minute >= 60) {
21         throw std::invalid_argument{"minute was out of range"};
22     }
23
24     if (second < 0 || second >= 60) {
25         throw std::invalid_argument{"second was out of range"};
26     }
27
28     m_hour = hour;
29     m_minute = minute;
30     m_second = second;
31 }
```

Time Class Case Study: Constructors with Default Arguments (Cont.)

- Class's Constructor

- Function setTime (lines 14–31) ensures that hour is in the range 0–23 and minute and second are each in the range 0–59.
 - ◆ This version of setTime uses separate if statements to validate the arguments so we can provide precise error messages, indicating which argument is out of range.

```
13 // set new Time value using 24-hour time
14 void Time::setTime(int hour, int minute, int second) {
15     // validate hour, minute and second
16     if (hour < 0 || hour >= 24) {
17         throw std::invalid_argument{"hour was out of range"};
18     }
19
20     if (minute < 0 || minute >= 60) {
21         throw std::invalid_argument{"minute was out of range"};
22     }
23
24     if (second < 0 || second >= 60) {
25         throw std::invalid_argument{"second was out of range"};
26     }
27
28     m_hour = hour;
29     m_minute = minute;
30     m_second = second;
31 }
```

Time Class Case Study: Constructors with Default Arguments (Cont.)

- Class's Constructor

- Functions `setHour`, `setMinute` and `setSecond` (lines 34–40) each call `setTime`.
 - ◆ Each passes its argument and the current values of the other two data members.
 - ◆ For example, to change `m_hour`, `setHour` passes its hour argument and the current values of `m_minute` and `m_second`.

```
33 // set hour value
34 void Time::setHour(int hour) {setTime(hour, m_minute, m_second);}
35
36 // set minute value
37 void Time::setMinute(int minute) {setTime(m_hour, minute, m_second);}
38
39 // set second value
40 void Time::setSecond(int second) {setTime(m_hour, m_minute, second);}
```

Time Class Case Study: Constructors with Default Arguments (Cont.)

- Class's Constructor

- The C++ Core Guidelines provide many constructor recommendations.
 - ◆ We'll see more in the next two chapters.
- If a class has a **class invariant**—that is, it requires its data members to have specific values or ranges of values (as in class `Time`)—the class should define a constructor that validates its arguments and, if any are invalid, throws an exception to prevent the object from being created.

Time Class Case Study: Constructors with Default Arguments (Cont.)

- Testing the Updated Class Time

- Function main initializes five Time objects:

- ◆ one with all three arguments defaulted in the implicit constructor call (line 16),
 - ◆ one with one argument specified (line 17),
 - ◆ one with two arguments specified (line 18),
 - ◆ one with three arguments specified (line 19) and
 - ◆ one with three invalid arguments specified (line 29).

```
15 int main() {
16     const Time t1{}; // all arguments defaulted
17     const Time t2{2}; // hour specified; minute & second defaulted
18     const Time t3{21, 34}; // hour & minute specified; second defaulted
19     const Time t4{12, 25, 42}; // hour, minute & second specified
20
21     std::cout << "Constructed with:\n\n";
22     displayTime("t1: all arguments defaulted", t1);
23     displayTime("t2: hour specified; minute and second defaulted", t2);
24     displayTime("t3: hour and minute specified; second defaulted", t3);
25     displayTime("t4: hour, minute and second specified", t4);
```

Time Class Case Study: Constructors with Default Arguments (Cont.)

- Testing the Updated Class Time

- The program displays each object in 24-hour and 12-hour time formats.
- The program displays an error message for Time object t5 (line 29) because its constructor arguments are out of range.
 - ◆ The variable t5 never represents a fully constructed object in this program because the exception is thrown during construction.

```
9 // displays a Time in 24-hour and 12-hour formats
10 void displayTime(std::string_view message, const Time& time) {
11     std::cout << std::format("{}\n24-hour time: {}\n12-hour time: {}\n\n",
12         message, time.to24HourString(), time.to12HourString());
13 }
14
27 // attempt to initialize t5 with invalid values
28 try {
29     const Time t5{27, 74, 99}; // all bad values specified
30 }
31 catch (const std::invalid_argument& e) {
32     std::cerr << std::format("t5 not created: {}\n", e.what());
33 }
34 }
```

Time Class Case Study: Constructors with Default Arguments (Cont.)

- Software Engineering Notes Regarding Class Time's Set and Get Functions and Its Constructor
 - Time's set and get functions are called throughout the class's body. In particular, the constructor (lines 9–11) calls setTime, and to24HourString and to12HourString call getHour, getMinute and getSecond in lines 54 and lines 60–61.
 - We could have accessed the class's private data in each case directly.

```
8 // Time constructor initializes each data member
9 Time::Time(int hour, int minute, int second) {
10     setTime(hour, minute, second);
11 }
51 // return Time as a string in 24-hour format (HH:MM:SS)
52 std::string Time::to24HourString() const {
53     return std::format("{:02d}:{:02d}:{:02d}",
54         getHour(), getMinute(), getSecond());
55 }
56
57 // return Time as a string in 12-hour format (HH:MM:SS AM or PM)
58 std::string Time::to12HourString() const {
59     return std::format("{}:{:02d}:{:02d} {}",
60         ((getHour() % 12 == 0) ? 12 : getHour() % 12),
61         getMinute(), getSecond(), (getHour() < 12 ? "AM" : "PM"));
62 }
```

Time Class Case Study: Constructors with Default Arguments (Cont.)

- Software Engineering Notes Regarding Class Time's Set and Get Functions and Its Constructor
 - Our internal time representation uses three ints, requiring 12 bytes of memory on systems with four-byte ints.
 - Consider changing this to the total seconds since midnight—a single int requiring only four bytes of memory. If we made this change, only the bodies of functions directly accessing the private data would need to change.
 - ◆ In this class, we'd modify setTime and the set and get function's bodies for m_hour, m_minute and m_second. There would be no need to modify the constructor or functions to24HourString or to12HourString because they do not access the data directly.

Time Class Case Study: Constructors with Default Arguments (Cont.)

- Software Engineering Notes Regarding Class Time's Set and Get Functions and Its Constructor
 - Duplicating statements in multiple functions or constructors makes changing the class's internal data representation more difficult.
 - ◆ Implementing the Time constructor and functions to24HourString and to12HourString, as shown in this example, reduces the likelihood of errors when altering the class's implementation.
 - As a general rule: **Avoid repeating code**. This principle is called **DRY**—"don't repeat yourself."
 - ◆ Rather than duplicating code, place it in a member function that can be called by the class's constructor or other member functions.
 - ◆ This simplifies code maintenance and reduces the likelihood of an error if the code implementation is modified.

Time Class Case Study: Constructors with Default Arguments (Cont.)

- Software Engineering Notes Regarding Class Time's Set and Get Functions and Its Constructor
 - A constructor can call the class's other member functions.
 - ◆ You must be careful when doing this.
 - ◆ The constructor initializes the object, so data members used in the called function may not yet be initialized.
 - ◆ Logic errors may occur if you use data members before properly initializing them.
 - Making data members private and controlling access (especially write access) to those data members through public member functions helps ensure data integrity.
 - ◆ The benefits of data integrity are not automatic simply because data members are private.
 - ◆ You must provide appropriate validity checking.

Time Class Case Study: Constructors with Default Arguments (Cont.)

- Overloaded Constructors and Delegating Constructors
 - Chapter 5 showed how to overload functions.
 - A class's constructors and member functions also can be overloaded.
 - Overloaded constructors allow objects to be initialized with different types and/or numbers of arguments.
 - To overload a constructor, provide a prototype and definition for each overloaded version.
 - This also applies to overloaded member functions.

Time Class Case Study: Constructors with Default Arguments (Cont.)

- Overloaded Constructors and Delegating Constructors
 - In the example, class Time's constructor had a default argument for each parameter.
 - We could have defined that constructor instead as four overloaded constructors with the following prototypes (and appropriate definitions in Time.cpp):
 - ◆ `Time(); // default m_hour, m_minute and m_second to 0`
 - ◆ `explicit Time(int hour); // default m_minute & m_second to 0`
 - ◆ `Time(int hour, int minute); // default m_second to 0`
 - ◆ `Time(int hour, int minute, int second); // no default values`

Time Class Case Study: Constructors with Default Arguments.

- Overloaded Constructors and Delegating Constructors
 - Just as a constructor can call a class's other member functions to perform tasks, **constructors can call other constructors in the same class.**
 - ◆ The calling constructor is called a **delegating constructor**—it delegates its work to another constructor.
 - ◆ The C++ Core Guidelines recommend defining common code for overloaded constructors in one constructor, then using delegating constructors to call it.
 - The first three of the four Time preceding constructor declarations can delegate work to one with three int arguments, passing 0 as the default value for the extra parameters.
 - To do so, you use a member initializer with the name of the class, as in:
 - ◆ `Time::Time() : Time{0, 0, 0} {}`
 - ◆ `Time::Time(int hour) : Time{hour, 0, 0} {}`
 - ◆ `Time::Time(int hour, int minute) : Time{hour, minute, 0} {}`

Destructors

- A destructor is a special member function that may **not specify parameters or a return type**.
- A class's destructor name is the **tilde character (~)**, **followed by the class name**, such as ~Time.
- A class's destructor is called implicitly when an object is destroyed, typically when program control leaves the scope in which that object was created.

Destructors.

- The destructor itself does not actually remove the object from memory.
 - It performs termination housekeeping (such as closing a file and cleaning up other resources used by the object) before the object's memory is reclaimed for later use.
- Even though destructors have not been defined for the classes presented so far, every class has **exactly one destructor**.
 - If you do not explicitly define a destructor, the compiler defines a default destructor that invokes any class-type data members' destructors.
 - In Chapter 12, we'll explain why exceptions should **NOT** be thrown from destructors.

When Constructors and Destructors Are Called

- The order in which constructors and destructors are called depends on the objects' scopes.
- Generally, **destructor calls are made in the reverse order of the corresponding constructor calls**, but as we'll see in the next example, global and static objects can alter the order in which destructors are called.

When Constructors and Destructors Are Called (Cont.)

- Constructors and Destructors for Objects in **Global Scope**
 - **Constructors** are called for objects defined in the global scope (also called global namespace scope) **before executing any other function** in that file.
 - The **execution order of global object** constructors among multiple files **is NOT guaranteed**, so global objects in separate files should not depend on one another.
 - When **main terminates** via a return statement or by reaching its closing brace, the corresponding **destructors are called in the reverse order of their construction**.

When Constructors and Destructors Are Called (Cont.)

- Destructors on Program Termination

- The **exit function** often is called to terminate a program immediately when a fatal unrecoverable error occurs.
 - ◆ In this case, the **destructors of local objects do not execute**.
- **Function abort** performs similarly to function exit but forces the program to terminate immediately without allowing programmer-defined cleanup code of any kind to be called.
 - ◆ Function abort is usually used to indicate abnormal program termination.

When Constructors and Destructors Are Called (Cont.)

- Constructors and Destructors for **Non-static Local Objects**

- A non-static local object's **constructor** is called when **execution reaches the object's definition**.
- Its **destructor** is called when **execution leaves the object's scope**—that is, when the block in which that object is defined finishes executing normally or due to an exception.
- Destructors are **not called** for local objects if the program terminates **with a call to function exit or abort**.

When Constructors and Destructors Are Called (Cont.)

- Constructors and Destructors for **static Local Objects**
 - The **constructor** for a static local object is called only once when **execution first reaches** the point where the object is **defined**.
 - The corresponding **destructor** is called when **main terminates** or the **program calls function exit**.
 - ◆ Global and static objects are destroyed in the reverse order of their creation.
 - ◆ Destructors are **not called** for static objects if the program terminates **with a call to abort**.

When Constructors and Destructors Are Called (Cont.)

- Demonstrating When Constructors and Destructors Are Called
 - The next program demonstrates the order in which constructors and destructors are called for global, local and local static objects of class CreateAndDestroy.
 - ◆ This mechanical example is purely for pedagogic purposes.
 - ◆ Most classes you create will not require custom destructors unless objects of the class manage dynamically allocated memory (discussed in Chapter 11).

When Constructors and Destructors Are Called (Cont.)

- Demonstrating When Constructors and Destructors Are Called
 - Lines 13–14 of header declare the class's data members—an integer (m_ID) and a string (m_message) to identify each object in the program's output.

```
1 // CreateAndDestroy.h
2 // CreateAndDestroy class definition.
3 // Member functions defined in CreateAndDestroy.cpp.
4 #pragma once // prevent multiple inclusions of header
5 #include <string>
6 #include <string_view>
7
8 class CreateAndDestroy {
9 public:
10     CreateAndDestroy(int ID, std::string_view message); // constructor
11     ~CreateAndDestroy(); // destructor
12 private:
13     int m_ID; // ID number for object
14     std::string m_message; // message describing object
15 };
```

When Constructors and Destructors Are Called (Cont.)

- Demonstrating When Constructors and Destructors Are Called
 - The constructor and destructor implementations both display lines of output to indicate when they're called.

```
1 // CreateAndDestroy.cpp
2 // CreateAndDestroy class member-function definitions.
3 #include <format>
4 #include <iostream>
5 #include "CreateAndDestroy.h" // include CreateAndDestroy class definition
6
7 // constructor sets object's ID number and descriptive message
8 CreateAndDestroy::CreateAndDestroy(int ID, std::string_view message)
9 : m_ID{ID}, m_message{message} {
10     std::cout << std::format("Object {} constructor runs {}\n",
11                             m_ID, m_message);
12 }
13
14 // destructor
15 CreateAndDestroy::~CreateAndDestroy() {
16     // output newline for certain objects; helps readability
17     std::cout << std::format("{}Object {} destructor runs {}\n",
18                             (m_ID == 1 || m_ID == 6 ? "\n" : ""), m_ID, m_message);
19 }
```

When Constructors and Destructors Are Called (Cont.)

- Demonstrating When Constructors and Destructors Are Called
 - In the destructor, the conditional expression (line 18) determines whether the object being destroyed has the m_ID value 1 or 6 and, if so, outputs a newline character to make the program's output easier to follow.

```
1 // CreateAndDestroy.cpp
2 // CreateAndDestroy class member-function definitions.
3 #include <format>
4 #include <iostream>
5 #include "CreateAndDestroy.h" // include CreateAndDestroy class definition
6
7 // constructor sets object's ID number and descriptive message
8 CreateAndDestroy::CreateAndDestroy(int ID, std::string_view message)
9 : m_ID{ID}, m_message{message} {
10     std::cout << std::format("Object {} constructor runs {}\n",
11                             m_ID, m_message);
12 }
13
14 // destructor
15 CreateAndDestroy::~CreateAndDestroy() {
16     // output newline for certain objects; helps readability
17     std::cout << std::format("{}Object {} destructor runs {}\n",
18                             (m_ID == 1 || m_ID == 6 ? "\n" : ""), m_ID, m_message);
19 }
```

When Constructors and Destructors Are Called (Cont.)

- Demonstrating When Constructors and Destructors Are Called
 - The main file defines the object first (line 9) in the global scope.
 - ◆ Its constructor is called before any statements in main execute, and its destructor is called at program termination after the destructors for all objects with automatic storage duration have run.

```
1 // main.cpp
2 // Order in which constructors and
3 // destructors are called.
4 #include <iostream>
5 #include "CreateAndDestroy.h" // include CreateAndDestroy class definition
6
7 void create(); // prototype
8
9 const CreateAndDestroy first{1, "(global before main)"}; // global object
```

When Constructors and Destructors Are Called (Cont.)

- Demonstrating When Constructors and Destructors Are Called
 - Function main (lines 11–21) defines three objects.
 - ◆ Objects second (line 13) and fourth (line 19) are local objects, and object third (line 14) is a static local object.
 - ◆ The constructor for each object is called when execution reaches the point where that object is defined.

```
11 int main() {  
12     std::cout << "\nMAIN FUNCTION: EXECUTION BEGINS\n";  
13     const CreateAndDestroy second{2, "(local in main)"};  
14     static const CreateAndDestroy third{3, "(local static in main)"};  
15  
16     create(); // call function to create objects  
17  
18     std::cout << "\nMAIN FUNCTION: EXECUTION RESUMES\n";  
19     const CreateAndDestroy fourth{4, "(local in main)"};  
20     std::cout << "\nMAIN FUNCTION: EXECUTION ENDS\n";  
21 }
```

When Constructors and Destructors Are Called (Cont.)

- Demonstrating When Constructors and Destructors Are Called
 - When execution reaches the end of main, the destructors for objects fourth then second are called in the reverse of their constructors' order.
 - ◆ Object third is static, so it exists until program termination.
 - ◆ The destructor for object third is called before the destructor for global object first, but after non-static local objects are destroyed.

```
11 int main() {  
12     std::cout << "\nMAIN FUNCTION: EXECUTION BEGINS\n";  
13     const CreateAndDestroy second{2, "(local in main)"};  
14     static const CreateAndDestroy third{3, "(local static in main)"};  
15  
16     create(); // call function to create objects  
17  
18     std::cout << "\nMAIN FUNCTION: EXECUTION RESUMES\n";  
19     const CreateAndDestroy fourth{4, "(local in main)"};  
20     std::cout << "\nMAIN FUNCTION: EXECUTION ENDS\n";  
21 }
```

When Constructors and Destructors Are Called (Cont.)

- Demonstrating When Constructors and Destructors Are Called
 - Function create (lines 24–30) defines three objects—fifth (line 26) and seventh (line 28) are local automatic objects, and sixth (line 27) is a static local object.

```
23 // function to create objects
24 void create() {
25     std::cout << "\nCREATE FUNCTION: EXECUTION BEGINS\n";
26     const CreateAndDestroy fifth{5, "(local in create)"};
27     static const CreateAndDestroy sixth{6, "(local static in create)"};
28     const CreateAndDestroy seventh{7, "(local in create)"};
29     std::cout << "\nCREATE FUNCTION: EXECUTION ENDS\n";
30 }
```

When Constructors and Destructors Are Called.

- Demonstrating When Constructors and Destructors Are Called
 - When create terminates, the destructors for objects seventh then fifth are called in the reverse of their constructors' order.
 - Because sixth is static, it exists until program termination. The destructor for sixth is called before the destructors for third and first, but after all other non-static objects are destroyed.
 - ◆ As an exercise, modify this program to call create twice—you'll see that the static object sixth's constructor is called once when create is called the first time.

```
23 // function to create objects
24 void create() {
25     std::cout << "\nCREATE FUNCTION: EXECUTION BEGINS\n";
26     const CreateAndDestroy fifth{5, "(local in create)"};
27     static const CreateAndDestroy sixth{6, "(local static in create)"};
28     const CreateAndDestroy seventh{7, "(local in create)"};
29     std::cout << "\nCREATE FUNCTION: EXECUTION ENDS\n";
30 }
```

A Subtle Trap—Returning a Reference or a Pointer to a private Data Member

- A reference to an object is an alias for the object's name, so it may be used on the left side of an assignment statement.
 - In this context, the reference makes a perfectly acceptable lvalue to which you can assign a value.
- A member function can return a reference to a private data member of that class.
 - If the reference return type is declared `const`, as we did for the `getName` member function of our `Account` class earlier in this chapter, the reference is a nonmodifiable lvalue and cannot be used to modify the data.
 - However, **subtle errors can occur if the reference return type is not declared `const`.**

A Subtle Trap—Returning a Reference or a Pointer to a private Data Member (Cont.)

- The next program is a mechanical example that uses a simplified Time class to demonstrate the risk of returning a reference to a private data member.

```
1 // main.cpp
2 // public member function that
3 // returns a reference to private data.
4 #include <iostream>
5 #include <format>
6 #include "Time.h" // include definition of class Time
7
8 int main() {
9     Time t{}; // create Time object
10
11     // initialize hourRef with the reference returned by badSetHour
12     int& hourRef{t.badSetHour(20)}; // 20 is a valid hour
13
14     std::cout << std::format(
15         "Valid hour before modification: {}\n", hourRef);
16     hourRef = 30; // use hourRef to set invalid value in Time object t
17     std::cout << std::format(
18         "Invalid hour after modification: {}\n\n", t.getHour());
19
20     // Dangerous: Function call that returns a reference can be
21     // used as an lvalue! POOR PROGRAMMING PRACTICE!!!!!!!
22     t.badSetHour(12) = 74; // assign another invalid value to hour
23
24     std::cout << "After using t.badSetHour(12) as an lvalue, "
25         << std::format("hour is: {}\n", t.getHour());
26 }
```

Valid hour before modification: 20

Invalid hour after modification: 30

After using t.badSetHour(12) as an lvalue, hour is: 74

A Subtle Trap—Returning a Reference or a Pointer to a private Data Member (Cont.)

- Member function `badSetHour` (declared in line 12 of header and defined in lines 30–33 of source code) returns an **`int&`** to the `m_hour` data member.
 - Such a reference return makes the result of a call to member function `badSetHour` an alias for private data member `hour`!
 - The function call can be used like the private data member as an lvalue in an assignment statement.
 - So, clients of the class can overwrite the class's private data at will!
 - A similar problem would occur if the function returned a pointer to the private data.

```
29 // poor practice: returning a reference to a private data member.
30 int& Time::badSetHour(int hour) {
31     setTime(hour, m_minute, m_second);
32     return m_hour; // dangerous reference return
33 }
```

A Subtle Trap—Returning a Reference or a Pointer to a private Data Member (Cont.)

- The main declares Time object t (line 9) and reference hourRef (line 12), which we initialize with the reference returned by t.badSetHour(20).
- Lines 14–15 display hourRef's value to show that hourRef breaks the class's encapsulation. Statements in main should not have access to the private data in a Time object.
- Next, line 16 uses the hourRef to set hour's value to the invalid value 30.
- Lines 17–18 call getHour to show that assigning to hourRef modified t's private data.

```
9      Time t{}; // create Time object
10
11     // initialize hourRef with the reference returned by badSetHour
12     int& hourRef{t.badSetHour(20)}; // 20 is a valid hour
13
14     std::cout << std::format(
15         "Valid hour before modification: {}\n", hourRef);
16     hourRef = 30; // use hourRef to set invalid value in Time object t
17     std::cout << std::format(
18         "Invalid hour after modification: {}\n\n", t.getHour());
```

Valid hour before modification: 20
Invalid hour after modification: 30

A Subtle Trap—Returning a Reference or a Pointer to a private Data Member (Cont.)

- Line 22 uses the `badSetHour` function call as an lvalue and assigns the invalid value 74 to the reference the function returns.
- Lines 24–25 call `getHour` again to show that line 22 modified the private data in the `Time` object `t`.

```
20 // Dangerous: Function call that returns a reference can be
21 // used as an lvalue! POOR PROGRAMMING PRACTICE!!!!!!!
22 t.badSetHour(12) = 74; // assign another invalid value to hour
23
24 std::cout << "After using t.badSetHour(12) as an lvalue, "
25 << std::format("hour is: {}\n", t.getHour());
26 }
```

After using `t.badSetHour(12)` as an lvalue, hour is: 74

A Subtle Trap—Returning a Reference or a Pointer to a private Data Member.

- Returning a reference or a pointer to a private data member breaks the class's encapsulation, making the client code dependent on the class's data representation.
- There are cases where doing this is appropriate. We'll show an example of this when we build our custom `MyArray` class in Chapter 11.

Default Assignment Operator

- The assignment operator (=) can assign an object to another object of the same type.
- The **default assignment operator** generated by the compiler **copies each data member of the right operand into the same data member in the left operand.**

Default Assignment Operator (Cont.)

- In the next example we define a Date class.
- Line 15 of main uses the default assignment operator to assign Date object date1 to Date object date2.
 - In this case, date1's m_year, m_month and m_day members are assigned to date2's m_year, m_month and m_day members, respectively.

```
1 // Date.h
2 // Date class declaration. Member functions are defined in Date.cpp.
3 #pragma once // prevent multiple inclusions of header
4 #include <string>
5
6 // class Date definition
7 class Date {
8 public:
9     Date(int year, int month, int day);
10    std::string toString() const;
11 private:
12    int m_year;
13    int m_month;
14    int m_day;
15 };
```

Default Assignment Operator (Cont.)

- Implementation of Date

- Lines 8 and 12 defines the constructor and toString member function.

```
1 // Date.cpp
2 // Date class member-function definitions.
3 #include <format>
4 #include <string>
5 #include "Date.h" // include definition of class Date from Date.h
6
7 // Date constructor (should do range checking)
8 Date::Date(int year, int month, int day)
9     : m_year{year}, m_month{month}, m_day{day} {}
10
11 // return string representation of a Date in the format yyyy-mm-dd
12 std::string Date::toString() const {
13     return std::format("{}-{:02d}-{:02d}", m_year, m_month, m_day);
14 }
```

Default Assignment Operator (Cont.)

- Test Driving Date

```
1  // main.cpp
2  // Demonstrating that class objects can be assigned
3  // to each other using the default assignment operator.
4  #include <format>
5  #include <iostream>
6  #include "Date.h" // include definition of class Date from Date.h
7  using namespace std;
8
9  int main() {
10     const Date date1{2006, 7, 4};
11     Date date2{2022, 1, 1};
12
13     std::cout << std::format("date1: {}\ndate2: {}\n\n",
14                             date1.toString(), date2.toString());
15     date2 = date1; // uses the default assignment operator
16     std::cout << std::format("After assignment, date2: {}\n",
17                             date2.toString());
18 }
```

Default Assignment Operator.

- Copy Constructors

- Objects may be passed as function arguments and may be returned from functions.
 - ◆ Such passing and returning are performed using **pass-by-value by default**—a copy of the object is passed or returned.
 - ◆ In such cases, C++ creates a new object and **uses a copy constructor to copy the original object's data** into the new object.
 - ◆ For each class we've shown so far, the compiler provides a **default copy constructor** that copies each member of the original object into the corresponding member of the new object.
 - ◆ For example, you can copy date1's data-member values into a new Date object with the statement:
`Date date3{date1};`

const Objects and const Member Functions

- Let's see how the principle of least privilege applies to objects. Some objects do not need to be modifiable, so you should declare them `const`. Any attempt to modify a `const` object results in a compilation error.
- The statement
`const Time noon{12, 0, 0};`
declares a **const Time object** `noon` and initializes it to 12 noon (12 PM). It's possible to instantiate `const` and non-`const` objects of the same class.
- C++ **disallows** calling a member function on a `const` object unless that member function is declared `const`.
 - So, **declare as `const` any member function that does not modify the object on which it's called.**

const Objects and const Member Functions (Cont.)

- A constructor must be allowed to modify an object to initialize it.
- A destructor must be allowed to perform its termination housekeeping before an object's memory is reclaimed by the system.
- So, declaring a constructor or destructor const is a compilation error.
- The “**constness**” of a const object is enforced throughout the object's lifetime **after the object is constructed**.

const Objects and const Member Functions (Cont.)

- Using const and Non-const Member Functions
 - The next program creates two Time objects—non-const object wakeUp (line 6) and const object noon (line 7).

```
1 // main.cpp
2 // const objects and const member functions.
3 #include "Time.h" // include Time class definition
4
5 int main() {
6     Time wakeUp{6, 45, 0}; // non-constant object
7     const Time noon{12, 0, 0}; // constant object
8
9     // OBJECT          MEMBER FUNCTION
10    wakeUp.setHour(18); // non-const non-const
11    noon.setHour(12);   // const    non-const
12    wakeUp.getHour();   // non-const const
13    noon.getMinute();   // const    const
14    noon.to24HourString(); // const    const
15    noon.to12HourString(); // const    non-const
16 }
```

const Objects and const Member Functions (Cont.)

- Using const and Non-const Member Functions
 - The program attempts to invoke non-const member functions `setHour` (line 11) and `to12HourString` (line 15) on the const object `noon`.
 - In each case, the compiler generates an error message.

9		// OBJECT	MEMBER FUNCTION
10	<code>wakeUp.setHour(18);</code>	// <u>non-const</u>	<u>non-const</u>
11	<code>noon.setHour(12);</code>	// <u>const</u>	<u>non-const</u>
12	<code>wakeUp.getHour();</code>	// <u>non-const</u>	<u>const</u>
13	<code>noon.getMinute();</code>	// <u>const</u>	<u>const</u>
14	<code>noon.to24HourString();</code>	// <u>const</u>	<u>const</u>
15	<code>noon.to12HourString();</code>	// <u>const</u>	<u>non-const</u>

const Objects and const Member Functions (Cont.)

- Using const and Non-const Member Functions
 - The program also illustrates the three other member-function-call combinations on objects:
 - ◆ a non-const member function on a non-const object (line 10),
 - ◆ a const member function on a non-const object (line 12) and
 - ◆ a const member function on a const object (lines 13–14).
 - The error messages generated for non-const member functions called on a const object are shown in the output window. We added blank lines for readability.

		// OBJECT	MEMBER FUNCTION
9			
10	wakeUp.setHour(18);	// non-const	non-const
11	noon.setHour(12);	// const	non-const
12	wakeUp.getHour();	// non-const	const
13	noon.getMinute();	// const	const
14	noon.to24HourString();	// const	const
15	noon.to12HourString();	// const	non-const

const Objects and const Member Functions.

- Using const and Non-const Member Functions

- A constructor must be a non-const member function, but it can still initialize a const object (line 7).

```
7 |    const Time noon{12, 0, 0}; // constant object
```

- Recall that the Time constructor's definition calls non-const member function setTime to initialize a Time object.
 - ◆ Invoking a non-const member function from the constructor for a const object is allowed.
 - ◆ Again, the object is not const until the constructor finishes initializing the object.
- Line 15 generates a compilation error even though Time's member function to12HourString does not modify the object on which it's called.
 - ◆ This fact is not sufficient—you must explicitly declare the function const for this call to be allowed by the compiler.

```
15 |    noon.to12HourString(); // const      non-const
```

Composition: Objects as Members of Classes

- An AlarmClock object needs to know when it's supposed to sound its alarm, so why not include a Time object as a member of the AlarmClock class?
- Such a software-reuse capability is called **composition** and is sometimes referred to as a **has-a relationship**—a class can have objects of other classes as members.
- You've already used composition in this chapter's Account class examples.
 - Class Account contained a string object as a data member.

Composition: Objects as Members of Classes (Cont.)

- You've seen how to pass arguments to a constructor.
- Now, let's see how a class's constructor can pass arguments to member-object constructors via member initializers.
- The next program uses classes Date and Employee to demonstrate composition. Class Employee's definition has private data members m_firstName, m_lastName, m_birthDate and m_hireDate.
 - Members m_birthDate and m_hireDate are objects of class Date, which has private data members m_year, m_month and m_day.

Composition: Objects as Members of Classes (Cont.)

- Date Class Definition

```
1 // Date.h
2 // Date class definition; member functions defined in Date.cpp
3 #pragma once // prevent multiple inclusions of header
4 #include <string>
5
6 class Date {
7     public:
8         static const int monthsPerYear{12}; // months in a year
9         Date(int year, int month, int day);
10        std::string toString() const; // date string in yyyy-mm-dd format
11        ~Date(); // implementation displays when destruction occurs
12    private:
13        int m_year; // any year
14        int m_month; // 1-12 (January-December)
15        int m_day; // 1-31 based on month
16
17        // utility function to check if day is proper for month and year
18        bool checkDay(int day) const;
19};
```

Composition: Objects as Members of Classes (Cont.)

- Date Class Implementation

- Line 11 defines the constructor of date using member initializers list.

```
1  // Date.cpp
2  // Date class member-function definitions.
3  #include <array>
4  #include <format>
5  #include <iostream>
6  #include <stdexcept>
7  #include "Date.h" // include Date class definition
8
9  // constructor confirms proper value for month; calls
10 // utility function checkDay to confirm proper value for day
11 Date::Date(int year, int month, int day)
12     : m_year{year}, m_month{month}, m_day{day} {
13     if (m_month < 1 || m_month > monthsPerYear) { // validate the month
14         throw std::invalid_argument{"month must be 1-12"};
15     }
16
17     if (!checkDay(day)) { // validate the day
18         throw std::invalid_argument{
19             "Invalid day for current month and year"};
20     }
21
22     // output Date object to show when its constructor is called
23     std::cout << std::format("Date object constructor: {}\n", toString());
24 }
```

Composition: Objects as Members of Classes (Cont.)

- Date Class Implementation

- Line 27 defines toString member function to display the date
- Line 32 defines the destructor to show when date object is destroyed.

```
26 // gets string representation of a Date in the form yyyy-mm-dd
27 □ std::string Date::toString() const {
28     return std::format("{}-{:02d}-{:02d}", m_year, m_month, m_day);
29 }
30
31 // output Date object to show when its destructor is called
32 □ Date::~~Date() {
33     std::cout << std::format("Date object destructor: {}\n", toString());
34 }
```

Composition: Objects as Members of Classes (Cont.)

- Date Class Implementation
 - Line 38 checks the validity of month days.

```
36 // utility function to confirm proper day value based on
37 // month and year; handles leap years, too
38 bool Date::checkDay(int day) const {
39     // we ignore element 0
40     static const std::array daysPerMonth{
41         0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
42
43     // determine whether testDay is valid for specified month
44     if (1 <= day && day <= daysPerMonth.at(m_month)) {
45         return true;
46     }
47
48     // February 29 check for leap year
49     if (m_month == 2 && day == 29 && (m_year % 400 == 0 ||
50         (m_year % 4 == 0 && m_year % 100 != 0))) {
51         return true;
52     }
53
54     return false; // invalid day, based on current m_month and m_year
55 }
```

Composition: Objects as Members of Classes (Cont.)

```
1 // Employee.h
2 // Employee class definition showing composition.
3 // Member functions defined in Employee.cpp.
4 #pragma once // prevent multiple inclusions of header
5 #include <string>
6 #include <string_view>
7 #include "Date.h" // include Date class definition
8
9 class Employee {
10 public:
11     Employee(std::string_view firstName, std::string_view lastName,
12             const Date& birthDate, const Date& hireDate);
13     std::string toString() const;
14     ~Employee(); // provided to confirm destruction order
15 private:
16     std::string m_firstName; // composition: member object
17     std::string m_lastName; // composition: member object
18     Date m_birthDate; // composition: member object
19     Date m_hireDate; // composition: member object
20 };
```

Composition: Objects as Members of

```
1 // Employee.cpp
2 // Employee class member-function definitions.
3 #include <format>
4 #include <iostream>
5 #include "Employee.h" // Employee class definition
6 using namespace std;
7
8 // constructor uses member initializer list to pass initializer
9 // values to constructors of member objects
10 Employee::Employee(std::string_view firstName, std::string_view lastName,
11     const Date& birthDate, const Date& hireDate)
12     : m_firstName{firstName}, m_lastName{lastName},
13     m_birthDate{birthDate}, m_hireDate{hireDate} {
14     // output Employee object to show when constructor is called
15     std::cout << std::format("Employee object constructor: {} {}\n",
16         m_firstName, m_lastName);
17 }
18
19 // gets string representation of an Employee object
20 std::string Employee::toString() const {
21     return std::format("{} {} Hired: {} Birthday: {}", m_lastName,
22         m_firstName, m_hireDate.toString(), m_birthDate.toString());
23 }
24
25 // output Employee object to show when its destructor is called
26 Employee::~Employee() {
27     cout << std::format("Employee object destructor: {}, {}\n",
28         m_lastName, m_firstName);
29 }
```

Composition: Objects as Members of Classes (Cont.)

- Employee Class Definition

```
1 // Employee.h
2 // Employee class definition showing composition.
3 // Member functions defined in Employee.cpp.
4 #pragma once // prevent multiple inclusions of header
5 #include <string>
6 #include <string_view>
7 #include "Date.h" // include Date class definition
8
9 class Employee {
10 public:
11     Employee(std::string_view firstName, std::string_view lastName,
12             const Date& birthDate, const Date& hireDate);
13     std::string toString() const;
14     ~Employee(); // provided to confirm destruction order
15 private:
16     std::string m_firstName; // composition: member object
17     std::string m_lastName; // composition: member object
18     Date m_birthDate; // composition: member object
19     Date m_hireDate; // composition: member object
20 };
```

Composition: Objects as Members of Classes (Cont.)

- Employee Constructor's Member-Initializer List
 - The Employee constructor's prototype (lines 11–12 of header) specifies that the constructor has four parameters (firstName, lastName, birthDate and hireDate).
 - In the constructor's definition (lines 12–13 of source code), the first two parameters are passed via member initializers to the string constructor for data members firstName and lastName.

```
9  class Employee {  
10 public:  
11     Employee(std::string_view firstName, std::string_view lastName,  
12             const Date& birthDate, const Date& hireDate);  
13     std::string toString() const;  
14     ~Employee(); // provided to confirm destruction order
```

Composition: Objects as Members of Classes (Cont.)

- Employee Constructor's Member-Initializer List
 - The last two are passed via member initializers to class Date's constructor for data members birthdate and hireDate.
 - The data members are constructed in the order that they're declared in class Employee, not in the order they appear in the member-initializer list.
 - ◆ Again, for clarity, the C++ Core Guidelines recommend listing the member initializers in the order they're declared in the class.

```
9  class Employee {  
10 public:  
11     Employee(std::string_view firstName, std::string_view lastName,  
12             const Date& birthDate, const Date& hireDate);  
13     std::string toString() const;  
14     ~Employee(); // provided to confirm destruction order
```

Composition: Objects as Members of Classes (Cont.)

- Date Class's Default Copy Constructor

- As you study class Date, notice it does not provide a constructor with a Date parameter.
- So, why can the Employee constructor's member-initializer brace initialize the m_birthDate and m_hireDate objects by passing Date objects to their constructors?
- As mentioned previously, the **compiler provides each class with a default copy constructor** that copies each data member of the constructor's argument into the corresponding member of the object being initialized.
 - ◆ Chapter 11 discusses how to define customized copy constructors.

Composition: Objects as Members of Classes (Cont.)

```
1 // main.cpp
2 // Demonstrating composition--an object with member objects.
3 #include <format>
4 #include <iostream>
5 #include "Date.h" // Date class definition
6 #include "Employee.h" // Employee class definition
7
8 int main() {
9     const Date birth{1987, 7, 24};
10    const Date hire{2018, 3, 12};
11    const Employee manager{"Aisha", "Khan", birth, hire};
12
13    std::cout << std::format("\n{}\n\n", manager.toString());
14 }
```

Date object constructor: 1987-07-24
Date object constructor: 2018-03-12
Employee object constructor: Aisha Khan

Khan, Aisha Hired: 2018-03-12 Birthday: 1987-07-24

Employee object destructor: Khan, Aisha
Date object destructor: 2018-03-12
Date object destructor: 1987-07-24
Date object destructor: 2018-03-12
Date object destructor: 1987-07-24

Composition: Objects as Members of Classes (Cont.)

- Testing Classes Date and Employee

```
1 // main.cpp
2 // Demonstrating composition--an object with member objects.
3 #include <format>
4 #include <iostream>
5 #include "Date.h" // Date class definition
6 #include "Employee.h" // Employee class definition
7
8 int main() {
9     const Date birth{1987, 7, 24};
10    const Date hire{2018, 3, 12};
11    const Employee manager{"Aisha", "Khan", birth, hire};
12
13    std::cout << std::format("\n{}\n\n", manager.toString());
14 }
```

Date object constructor: 1987-07-24
Date object constructor: 2018-03-12
Employee object constructor: Aisha Khan

Khan, Aisha Hired: 2018-03-12 Birthday: 1987-07-24

Employee object destructor: Khan, Aisha
Date object destructor: 2018-03-12
Date object destructor: 1987-07-24
Date object destructor: 2018-03-12
Date object destructor: 1987-07-24

Composition: Objects as Members of Classes (Cont.)

- Testing Classes Date and Employee

- The main creates two Date objects (lines 9–10), then passes them as arguments to the constructor of the Employee object created in line 11.
- When an Employee is created, its constructor makes
 - ◆ two calls to the string class's constructor (line 12 of source code) and
 - ◆ two calls to the Date class's default copy constructor (line 13 of source code).
- Line 13 displays the Employee's data to show that it was initialized correctly.

```
8  int main() {  
9      const Date birth{1987, 7, 24};  
10     const Date hire{2018, 3, 12};  
11     const Employee manager{"Aisha", "Khan", birth, hire};  
12  
13     std::cout << std::format("\n{}\n\n", manager.toString());  
14 }
```

Composition: Objects as Members of Classes (Cont.)

- Testing Classes Date and Employee

- When each Date object is created in lines 9–10, the Date constructor (lines 11–24) displays a line of output to show that the constructor was called (see the first two lines of the sample output).
- However, line 11 of main causes two Date copy-constructor calls (line 13 of source code) that do not appear in this program's output.
 - ◆ Since the compiler defines our Date class's copy constructor, it does not contain any output statements to demonstrate when it's called.

```
8  int main() {  
9      const Date birth{1987, 7, 24};  
10     const Date hire{2018, 3, 12};  
11     const Employee manager{"Aisha", "Khan", birth, hire};  
12  
13     std::cout << std::format("\n{}\n\n", manager.toString());  
14 }
```

Composition: Objects as Members of Classes (Cont.)

- Testing Classes Date and Employee
 - Class Date and class Employee each include a destructor (lines 32–34 of Date’s source code and lines 26–29 of Employee’s source code, respectively) that prints a message when an object of its class is destructed.
 - ◆ The destructors help us show that objects are destructed from the outside in.
 - ◆ The Date member objects are destructed after the enclosing Employee object.

Composition: Objects as Members of Classes (Cont.)

- Testing Classes Date and Employee

- Notice the last four lines in the output of main.

- ◆ The last two lines are the outputs of the Date destructor running on Date objects hire (line 10) and birth (line 9), respectively.
 - ◆ The outputs confirm that the three objects created in main are destructed in the reverse order of their construction.
 - The Employee destructor output is five lines from the bottom.
 - The fourth and third lines from the bottom of the output show the destructors running for the Employee's member objects m_hireDate and m_birthDate (lines 18-19 of header).

```
Date object destructor: 2018-03-12
Date object destructor: 1987-07-24
Date object destructor: 2018-03-12
Date object destructor: 1987-07-24
```

Composition: Objects as Members of Classes (Cont.)

- Testing Classes Date and Employee
 - These outputs confirm that the Employee object is destructed from the outside in.
 - ◆ The Employee destructor runs first (see the output five lines from the bottom).
 - ◆ Then the member objects are destructed in the reverse order from which they were constructed.
 - ◆ Class string's destructor does not contain output statements, so we do not see the firstName and lastName objects being destructed.

Composition: Objects as Members of Classes.

- What Happens When You Do Not Use the Member-Initializer List?
 - If you do not initialize a member object explicitly, its default constructor will be called implicitly to initialize it.
 - If there is no default constructor, a compilation error occurs.
 - Values set by the default constructor can be changed later—for example, by set functions.
 - ◆ However, this approach may require significant additional work and time for complex initialization.
 - ◆ Initializing member objects via [member initializers](#) eliminates the overhead of “doubly initializing” member objects.

friend Functions and friend Classes

- A friend function has access to a class's public and non-public members. A class may have as friends
 - stand-alone functions,
 - entire classes (and thus all their functions) or
 - specific member functions of other classes.
- We present a mechanical example of how a friend function works.
- In Chapter 11, Operator Overloading, Copy/Move Semantics and Smart Pointers, we'll show friend functions that overload operators for use with objects of custom classes.
 - You'll see that sometimes a member function cannot be used to define certain overloaded operators.

friend Functions and friend Classes (Cont.)

- Declaring a friend

- To declare a non-member function as a friend of a class, place the function prototype in the class definition and precede it with the keyword friend.
- To declare all member functions of an existing class ClassTwo as friends of class ClassOne, place in ClassOne's definition a declaration of the form:
friend class ClassTwo;

friend Functions and friend Classes (Cont.)

- Friendship Rules

- These are the basic friendship rules:

- ◆ **Friendship is granted**, not taken—For class B to be a friend of class A, class A must declare that class B is its friend.
 - ◆ **Friendship is not symmetric**—If class A is a friend of class B, you cannot infer that class B is a friend of class A.
 - ◆ **Friendship is not transitive**—If class A is a friend of class B and class B is a friend of class C, you cannot infer that class A is a friend of class C.

friend Functions and friend Classes (Cont.)

- friends Are Not Subject to Access Modifiers
 - Member access notions of public, protected (Chapter 10) and private do not apply to friend declarations,
 - ◆ so friend declarations can be placed anywhere in a class definition.
 - ◆ We prefer to place friend declarations first inside the class definition's body and **not precede them with any access specifier**.

friend Functions and friend Classes (Cont.)

- Modifying a Class's private Data with a friend Function
 - The next program defines the free function modify as a friend of class Count (line 8), so modify can set Count's private data member m_x.

```
1 // main.cpp
2 // Friends can access private members of a class.
3 #include <format>
4 #include <iostream>
5 #include "fmt/format.h" // In C++20, this will be #include <format>
6
7 // Count class definition
8 class Count {
9     friend void modifyX(Count& c, int value); // friend declaration
10 public:
11     int getX() const { return m_x; }
12 private:
13     int m_x{0};
14 };
15
16 // function modifyX can modify private data of Count
17 // because modifyX is declared as a friend of Count (line 8)
18 void modifyX(Count& c, int value) {
19     c.m_x = value; // allowed because modifyX is a friend of Count
20 }
```

friend Functions and friend Classes (Cont.)

- Modifying a Class's private Data with a friend Function
 - Function modifyX (lines 17–19) is a stand-alone (free) function.
 - Function modifyX can access class Count's private data member m_x (line 18) only because the function was declared as a friend of class Count (line 8).
 - If you remove the friend declaration, you'll receive error messages indicating that function modifyX cannot access class Count's private data member m_x.

```
16 // function modifyX can modify private data of Count
17 // because modifyX is declared as a friend of Count (line 8)
18 void modifyX(Count& c, int value) {
19     c.m_x = value; // allowed because modifyX is a friend of Count
20 }
```

friend Functions and friend Classes.

- Modifying a Class's private Data with a friend Function
 - Function modifyX (lines 17–19) is not a Count member function.
 - So, when we call modifyX in main to modify the Count object counter (line 25), we must pass counter as an argument to modifyX.

```
22 int main() {  
23     Count counter{}; // create Count object  
24  
25     std::cout << std::format("Initial counter.m_x: {}\n", counter.getX());  
26     modifyX(counter, 8); // change x's value using a friend function  
27     std::cout << std::format("counter.m_x after modifyX: {}\n",  
28         counter.getX());  
29 }
```

```
Initial counter.m_x: 0  
counter.m_x after modifyX: 8
```

The this Pointer

- There's only one copy of each class's functionality, but there can be many objects of a class.
- So, how do member functions know which object's data members to manipulate?
- Every object's member functions access the object **through a pointer called this** (a C++ keyword), which is initialized with an **implicit argument passed to** each of the object's **non-static member functions**.

The this Pointer (Cont.)

- Using the this Pointer to Avoid Naming Collisions
 - Member functions use the this pointer implicitly (as we've done so far) or explicitly to reference an object's data members and other member functions.
 - One explicit use of the this pointer is to **avoid naming conflicts** between a class's data members and constructor or member-function parameters.
 - ◆ If a member function uses a local variable and a data member with the same name, the **local variable is said to hide** or shadow the **data member**.
 - Using just the variable name in the member function's body refers to the local variable rather than the data member.

The this Pointer (Cont.)

- Using the this Pointer to Avoid Naming Collisions
 - You can access the data member explicitly by qualifying its name with `this->`.
 - For instance, if class `Account` had a string data member `name`, we could implement its `setName` function as follows:

```
void Account::setName(std::string_view name) {  
    this->name = name; // use this-> to access data  
    member  
}
```

 - ◆ where `this->name` represents a data member called `name`.
 - ◆ You can avoid such naming collisions by naming your data members differently—for example, using the `"m_"` prefix, as shown in our classes so far.

The this Pointer (Cont.)

- Type of the this Pointer

- The this pointer's type depends on the **object's type and whether** the member function in which this is used is **declared const**:
 - ◆ In a non-const member function of class Time, the this pointer is a Time*—a pointer to a Time object.
 - ◆ In a const member function, this is a const Time*—a pointer to a Time constant.

The this Pointer (Cont.)

- Implicitly and Explicitly Using the this Pointer to Access an Object's Data Members
 - The next program is a mechanical example that demonstrates implicit and explicit use of the this pointer in a member function to display the private data `m_x` of a `Test` object.
 - Later, we will show some substantial and subtle examples of using `this`.

```
1  // main.cpp
2  // Using the this pointer to refer to object members.
3  #include <format>
4  #include <iostream>
5
6  class Test {
7  public:
8      explicit Test(int value);
9      void print() const;
10 private:
11     int m_x{0};
12 };
13
14 // constructor
15 Test::Test(int value) : m_x{value} {} // initialize x to value
```

The this Pointer (Cont.)

- Implicitly and Explicitly Using the this Pointer to Access an Object's Data Members
 - For illustration purposes, member function print (lines 19–30) first displays m_x using the this pointer implicitly (line 21)—only the data member's name is specified.

```
17 // print x using implicit then explicit this pointers;
18 // the parentheses around *this are required due to precedence
19 void Test::print() const {
20     // implicitly use the this pointer to access the member x
21     std::cout << std::format("      m_x = {}\n", m_x);
22
23     // explicitly use the this pointer and the arrow operator
24     // to access the member x
25     std::cout << std::format("  this->m_x = {}\n", this->m_x);
26
27     // explicitly use the dereferenced this pointer and
28     // the dot operator to access the member x
29     std::cout << std::format("(*this).m_x = {}\n", (*this).m_x);
30 }
```

```
m_x = 12
this->m_x = 12
(*this).m_x = 12
```

The this Pointer (Cont.)

- Implicitly and Explicitly Using the this Pointer to Access an Object's Data Members
 - Then print uses two different notations to access m_x through the this pointer:
 - ◆ this->m_x (line 25) and
 - ◆ (*this).m_x (line 29).
 - The parentheses around *this (line 29) are required because the dot operator (.) has higher precedence than the * pointer-dereferencing operator.
 - Without the parentheses, the expression *this.m_x would be evaluated as *(this.m_x). The dot operator cannot be used with a pointer, so this would be a compilation error.

```
32 int main() {  
33     const Test testObject{12}; // instantiate and initialize testObject  
34     testObject.print();  
35 }
```

The this Pointer (Cont.)

- Using the this Pointer to Enable Cascaded Function Calls
 - Another use of the this pointer is to **enable cascaded member-function calls**—that is, invoking multiple functions sequentially in the same statement, as you'll see in line 11 of the main in the next example.

```
1 // main.cpp
2 // Cascading member-function calls with the this pointer.
3 #include <format>
4 #include <iostream>
5 #include "Time.h" // Time class definition
6 using namespace std;
7
8 int main() {
9     Time t{}; // create Time object
10
11     t.setHour(18).setMinute(30).setSecond(22); // cascaded function calls
12
13     // output time in 24-hour and 12-hour formats
14     std::cout << std::format("24-hour time: {}\n12-hour time: {}\n\n",
15                             t.to24HourString(), t.to12HourString());
16
17     // cascaded function calls
18     std::cout << std::format("New 12-hour time: {}\n",
19                             t.setTime(20, 20, 20).to12HourString());
20 }
```

24-hour time: 18:30:22
12-hour time: 6:30:22 PM

New 12-hour time: 8:20:20 PM

The this Pointer (Cont.)

- Using the this Pointer to Enable Cascaded Function Calls
 - The program modifies class Time's setTime, setHour, setMinute and setSecond functions such that each **returns a reference to the Time object** on which it's called.
 - This reference enables cascaded member-function calls.
 - The last statement in setTime's body **returns a reference to *this** (line 30).
 - Functions setHour, setMinute and setSecond each call setTime and return its Time& result.

```
12 // set new Time value using 24-hour time
13 Time& Time::setTime(int hour, int minute, int second) {
    30     return *this; // enables cascading
    31 }
```

The this Pointer (Cont.)

- Using the this Pointer to Enable Cascaded Function Calls
 - Why does the technique of returning *this as a reference work?
 - The dot operator (.) groups left-to-right,
 - ◆ so line 11
t.setHour(18).setMinute(30).setSecond(22);
first evaluates t.setHour(18), which returns a reference to (the updated) object t as the value of this function call.
 - ◆ The remaining expression is then interpreted as
t.setMinute(30).setSecond(22);
 - ◆ The t.setMinute(30) call executes and returns a reference to the (further updated) object t.
 - ◆ The remaining expression is interpreted as
t.setSecond(22);

The this Pointer.

- Using the this Pointer to Enable Cascaded Function Calls
 - Line 19 also uses cascading.
 - ◆ We cannot chain another Time member-function call after `to12HourString`, because it does not return a `Time&`.
 - ◆ However, we could chain a call to a string member function because `to12HourString` returns a string.
 - Chapter 11 presents practical examples of cascaded function calls, such as in `cout` statements with the `<<` operator and in `cin` statements with the `>>` operator.

```
17 // cascaded function calls
18 std::cout << std::format("New 12-hour time: {}\n",
19     t.setTime(20, 20, 20).to12HourString());
20 }
```

static Class Members: Classwide Data and Member Functions

- There is an **exception** to the rule that **each object has its own copy of its class's data members**.
- In some cases, all objects of a class should **share only one copy** of a variable.
- A static data member is used for these and other reasons.
- Such a variable represents “**classwide**” information—that is, data shared by all objects of the class.
- You can use static data members to **save storage** when all objects of a class can share a single copy of the data.

static Class Members: Classwide Data and Member Functions (Cont.)

- Motivating Classwide Data
 - Let's further explore the need for static classwide data with an example.
 - Suppose that we have a video game with Martians and other SpaceCreatures.
 - Each Martian tends to be brave and willing to attack other SpaceCreatures when the Martian is aware that at least five Martians are present.
 - ◆ If fewer than five are present, each Martian becomes cowardly.
 - ◆ So each Martian needs to know the martianCount.

static Class Members: Classwide Data and Member Functions (Cont.)

- Motivating Classwide Data
 - We could endow each object of class Martian with **martianCount as a data member**.
 - ◆ If we do, **every Martian will have its own copy** of the data member.
 - ◆ Every time we create a new Martian, we'd have to update the data member martianCount in all Martian objects.
 - Doing this would require every Martian object to know about all other Martian objects in memory.
 - ◆ This **wastes space** with redundant martianCount copies **and wastes time** updating the separate copies.

static Class Members: Classwide Data and Member Functions (Cont.)

- Motivating Classwide Data

- Instead, we **declare martianCount to be static** to make it classwide data.
 - ◆ Every Martian can access martianCount as if it were a data member, but **only one copy** of the static martianCount is **maintained** in the program.
 - This **saves space**.
 - ◆ We have the Martian **constructor increment** static variable martianCount and the Martian **destructor decrement** martianCount.
 - Because there's only one copy, we do not have to increment or decrement separate copies of martianCount for every Martian object.

static Class Members: Classwide Data and Member Functions (Cont.)

- Scope and Initialization of static Data Members
 - A class's **static data members** have **class scope**.
 - A static data member must be **initialized exactly once**.
 - **Fundamental-type static** data members are **initialized** by default to 0.
 - A **static const** data member can **have an in-class initializer**.
 - You also may use in-class initializers **for a non-const static data member** **by preceding its declaration with the inline keyword**.
 - If a static data member is an object of a class that provides a default constructor, the static data member need not be explicitly initialized because its default constructor will be called.

static Class Members: Classwide Data and Member Functions (Cont.)

- Accessing static Data Members
 - A class's **static members exist even when no objects of that class exist.**
 - To access a public static class data member or member function, **simply prefix the class name and the scope resolution operator (::) to the member name.**
 - For example, if our `martianCount` variable is public, it can be accessed with `Martian::martianCount`, even when there are no `Martian` objects.
 - A class's private (and protected; Chapter 10) **static members** are normally **accessed through** the class's **public member functions or friends**.
 - ◆ To access a private static or protected static data member when no objects of the class exist, provide a public static member function and call the function by prefixing its name with the class name and scope resolution operator.
 - **A static member function is a service of the class as a whole, not of a specific object of the class.**

static Class Members: Classwide Data and Member Functions (Cont.)

- Demonstrating static Data Members

- This example demonstrates

- ◆ a private inline static data member called `m_count`, which is initialized to 0 (line 22), and
 - ◆ a public static member function called `getCount` (line 16).

```
8  class Employee {
9  public:
10     Employee(std::string_view firstName, std::string_view lastName);
11     ~Employee(); // destructor
12     const std::string& getFirstName() const; // return first name
13     const std::string& getLastName() const; // return last name
14
15     // static member function
16     static int getCount(); // return # of objects instantiated
17 private:
18     std::string m_firstName;
19     std::string m_lastName;
20
21     // static data
22     inline static int m_count{0}; // number of objects instantiated
23 };
```

static Class Members: Classwide Data and Member Functions (Cont.)

- Demonstrating static Data Members
 - A static data member also can be **initialized at file scope in the class's implementation file**.
 - ◆ For instance, we could have placed the following statement in Employee.cpp after the Employee.h header is included:
int Employee::count{0};

static Class Members: Classwide Data and Member Functions (Cont.)

- Demonstrating static Data Members

- In the Employee.cpp, line 10 defines static member function getCount—note this does **NOT** include the **static** keyword, which cannot be applied to a member definition that appears outside the class definition.

```
1 // Employee.cpp
2 // Employee class member-function definitions.
3 #include <format>
4 #include <iostream>
5 #include "Employee.h" // Employee class definition
6 using namespace std;
7
8 // define static member function that returns number of
9 // Employee objects instantiated (declared static in Employee.h)
10 int Employee::getCount() {return m_count;}
```

static Class Members: Classwide Data and Member Functions (Cont.)

- Demonstrating static Data Members

- In this program, data member `m_count` maintains a count of the number of `Employee` objects in memory at a given time.
- When `Employee` objects exist, member `m_count` can be referenced through any member function of an `Employee` object, as shown in the constructor (line 16) and the destructor (line 25).

```
12 // constructor initializes non-static data members and
13 // increments static data member count
14 Employee::Employee(string_view firstName, string_view lastName)
15 {
16     : m_firstName(firstName), m_lastName(lastName) {
17         ++m_count; // increment static count of employees
18         std::cout << std::format("Employee constructor called for {} {}\n",
19                                 m_firstName, m_lastName);
20     }
21 // destructor decrements the count
22 Employee::~Employee() {
23     std::cout << std::format("~Employee() called for {} {}\n",
24                             m_firstName, m_lastName);
25     --m_count; // decrement static count of employees
26 }
```

static Class Members: Classwide Data and Member Functions (Cont.)

- Demonstrating static Data Members
 - The main function uses static member function getCount to determine the number of Employee objects in memory at various points in the program.

```
7 int main() {
8     // no objects exist; use class name and scope resolution
9     // operator to access static member function getCount
10    std::cout << std::format("Initial employee count: {}\n",
11                             Employee::getCount()); // use class name
12
13    // the following scope creates and destroys
14    // Employee objects before main terminates
15    {
16        const Employee e1{"Susan", "Baker"};
17        const Employee e2{"Robert", "Jones"};
18
19        // two objects exist; call static member function getCount again
20        // using the class name and the scope resolution operator
21        std::cout << std::format(
22            "Employee count after creating objects: {}\n\n",
23            Employee::getCount());
24
25        std::cout << std::format("Employee 1: {} {}\nEmployee 2: {} {}\n\n",
26                                e1.getFirstName(), e1.getLastName(),
27                                e2.getFirstName(), e2.getLastName());
28    }
29
30    // no objects exist, so call static member function getCount again
31    // using the class name and the scope resolution operator
32    std::cout << std::format(
33        "Employee count after objects are destroyed: {}\n",
34        Employee::getCount());
35 }
```

static Class Members: Classwide Data and Member Functions (Cont.)

- Demonstrating static Data Members
 - The program calls `Employee::getCount()`:
 - ◆ before any `Employee` objects have been created (line 11),
 - ◆ after two `Employee` objects have been created (line 23) and
 - ◆ after those `Employee` objects have been destroyed (line 34).
 - When `Employee` objects are in scope, you also can call `getCount` on those objects.
 - ◆ For example, in line 23, we could have written either `e1.getCount()` or `e2.getCount()`.
 - ◆ Either would return the current value of class `Employee`'s static `m_count`.

static Class Members: Classwide Data and Member Functions (Cont.)

- Demonstrating static Data Members

- Lines 15–28 in main define a nested scope.
 - ◆ Recall that local variables exist until the scope in which they're defined terminates.
- In this example, we create two Employee objects in the nested scope (lines 16–17).
 - ◆ As each constructor executes, it increments class Employee's static data member m_count.

```
13 // the following scope creates and destroys
14 // Employee objects before main terminates
15 {
16     const Employee e1{"Susan", "Baker"};
17     const Employee e2{"Robert", "Jones"};
18
19     // two objects exist; call static member function getCount again
20     // using the class name and the scope resolution operator
21     std::cout << std::format(
22         "Employee count after creating objects: {}\\n\\n",
23         Employee::getCount());
24
25     std::cout << std::format("Employee 1: {} {}\\nEmployee 2: {} {}\\n\\n",
26         e1.getFirstName(), e1.getLastName(),
27         e2.getFirstName(), e2.getLastName());
28 }
```

static Class Members: Classwide Data and Member Functions (Cont.)

- Demonstrating static Data Members
 - These Employee objects are destroyed when the program reaches line 28.
 - ◆ At that point, each object's destructor executes and decrements class Employee's static data member m_count.

```
13 // the following scope creates and destroys
14 // Employee objects before main terminates
15 {
16     const Employee e1{"Susan", "Baker"};
17     const Employee e2{"Robert", "Jones"};
18
19     // two objects exist; call static member function getCount again
20     // using the class name and the scope resolution operator
21     std::cout << std::format(
22         "Employee count after creating objects: {} \n\n",
23         Employee::getCount());
24
25     std::cout << std::format("Employee 1: {} {} \nEmployee 2: {} {} \n\n",
26         e1.getFirstName(), e1.getLastName(),
27         e2.getFirstName(), e2.getLastName());
28 }
```

static Class Members: Classwide Data and Member Functions (Cont.)

- static Member Function Notes

- A member function should be declared static if it does not access the class's non-static data members or non-static member functions.
- A static member function does not have a this pointer because static data members and static member functions exist independently of any objects of a class.
 - ◆ The this pointer must refer to a specific object, but a static member function can be called when there are no objects of its class in memory.
 - ◆ So, using the this pointer in a static member function is a compilation error.

static Class Members: Classwide Data and Member Functions.

- static Member Function Notes

- A static member function cannot be declared const.

- ◆ The const qualifier applies to the object the this pointer points to and indicates that a function cannot modify the contents of that object.
 - ◆ However, static member functions exist and operate independently of any objects of the class, so declaring a static member function const is a compilation error.

Aggregates in C++20

- Section 9.4.1 of the C++ standard document (<http://wg21.link/n4861>) describes an aggregate type as a built-in array, an array object or an object of a class that
 - does not have user-declared constructors,
 - does not have private or protected (Chapter 10) non-static data members,
 - does not have virtual functions (Chapter 10) and
 - does not have private (Chapter 10), protected (Chapter 10) or virtual (Chapter 10) base classes.
- You can define an aggregate using a class in which **all the data is public**.

Aggregates in C++20 (Cont.)

- However, a struct is a class that contains only public members by default.
- The following struct defines an aggregate type named Record containing four public data members:

```
struct Record {  
    int account;  
    string first;  
    string last;  
    double balance;  
};
```
- The C++ Core Guidelines recommend **using class rather than struct if any data member or member function needs to be non-public.**

Aggregates in C++20 (Cont.)

- Initializing an Aggregate

- You can initialize an object of aggregate type Record (above) as follows:

`Record record{100, "Abilio", "Saiu", 123.45};`

- If you initialize an aggregate-type object with fewer initializers than there are data members in the object, as in

`Record record{0, "Abilio", "Saiu"};`

the remaining data members are initialized as follows:

- ◆ Data members **with in-class initializers use those values**—in the preceding case, record's balance is set to 0.0.
- ◆ Data members **without in-class initializers are initialized with empty braces {}**.
 - Empty-brace initialization sets fundamental-type variables to 0, sets bools to false and value initializes objects.

Aggregates in C++20 (Cont.)

- C++20 Designated Initializers

- As of C++20, aggregates now support **designated initializers** in which you can **specify which data members to initialize by name**.

- Using the struct Record definition, we could initialize a Record object as follows:

`Record record{.first{"Couro"}, .last{"Malonga"}};`

which explicitly initializes only a subset of the data members.

- ◆ Each explicitly named data member is preceded by a dot (.).
- The identifiers you specify must be **listed in the same order as they're declared** in the aggregate type.
 - ◆ The preceding statement initializes the data members first and last to "Couro" and "Malonga", respectively.
 - ◆ The remaining data members get their default initializer values:
 - account is set to 0 and
 - balance is set to its default value in the type definition—in this case, 0.0.

Aggregates in C++20.

- Other Benefits of Designated Initializers
 - Adding new data members to an aggregate type will not break existing statements that use designated initializers.
 - Any new data members that are not explicitly initialized simply receive their default initialization.
 - Designated initializers also [improve compatibility with the C programming language](#), which has had this feature since C99.

Studying the Vigenère Secret-Key Cipher Implementation

- In the preceding chapters, we presented nine Objects Natural case studies in which you were able to perform powerful tasks without knowing how to build custom classes.
- Here, we'll present the Vigenère secret-key cipher algorithm and study our Cipher class that implements the algorithm.
- You'll see that the class contains 171 lines of code, including blank lines and comments.
 - This class took significant effort for us to design, write, test and debug.

Studying the Vigenère Secret-Key Cipher Implementation (Cont.)

- Yet, you were able to use this implementation in Chapter 5 to conveniently explore a nice example of private-key cryptography.
- So you've learned that an important part of becoming a C++ developer is to “[avoid reinventing the wheel](#)”—take advantage of the enormous numbers of class libraries that the programming community has developed over C++'s four decades.
- A huge number of these are free and open source and available to you on sites like GitHub. Each repository contains a library's, project's or application's source files, and C++ library repositories often contain many classes.

Studying the Vigenère Secret-Key Cipher Implementation (Cont.)

- You've also worked with various existing classes from the C++ Standard Library, such as `string`, `array`, `vector`, `span` and `string_view`, most of which were included in our Objects Natural case studies.
- In Chapters 13–20, we'll tour other key aspects of the C++ Standard Library, including:
 - Chapter 11 and Chapter 20—Smart pointers which provide additional functionality beyond that of built-in pointers to help you avoid errors.
 - Chapter 12—Exceptions.
 - Chapter 13—Standard Library containers and iterators.
 - Chapter 14—Standard Library algorithms and C++20 ranges and views.
 - Chapter 15—C++20 Concepts and various class and function templates used in compile-time programming with templates.
 - Chapters 17 and 18—C++ prepackaged parallel algorithms, and concurrency and C++20 Coroutines capabilities for building programs that can run faster (often much faster) on today's multi-core computer architectures. We also demonstrate some of C++'s `<chrono>` library features to profile the performance improvement.
 - Chapter 19—Input/output streams and C++20 text formatting.
 - Chapter 20—More smart pointers, additional compile-time programming capabilities and more.

Studying the Vigenère Secret-Key Cipher Implementation (Cont.)

- Using Our Cipher Class

- The next program contains a modified version of the main application presented in Chapter 5 that demonstrates class Cipher with the plaintext "Welcome to encryption".

```
1 // main.cpp
2 // Encrypting and decrypting text with a Vigenere Secret-Key Cipher.
3 #include "cipher.h"
4 #include <iostream>
5 #include <string>
6
7 int main() {
8     std::string plainText;
9     std::cout << "Enter the text to encrypt:\n";
10    std::getline(std::cin, plainText);
11
12    std::string secretKey;
13    std::cout << "\nEnter the secret key:\n";
14    std::getline(std::cin, secretKey);
15
16    Cipher cipher;
17
18    // encrypt plainText using secretKey
19    std::string cipherText{cipher.encrypt(plainText, secretKey)};
20    std::cout << "\nEncrypted:\n    " << cipherText << '\n';
21
22    // decrypt cipherText
23    std::cout << "\nDecrypted:\n    "
24    << cipher.decrypt(cipherText, secretKey) << '\n';
25 }
```

Enter the text to encrypt:
Welcome to encryption

Enter the secret key:
XMWUJBVYHXZ

Encrypted:
Tqhwxnz rv bmzdujcjjl

Decrypted:
Welcome to encryption

Studying the Vigenère Secret-Key Cipher Implementation (Cont.)

- Class Cipher

- The header defines the Cipher class, which we separated into parts for discussion purposes.
- The first part (lines 1–14) shows the headers our Vigenère implementation requires and the beginning of the class definition.

```
1 // cipher.h
2 // Vigenere Secret-Key Cipher Implementation.
3 #pragma once
4 #include <algorithm>
5 #include <array>
6 #include <iostream>
7 #include <string>
8 #include <string_view>
9 #include <cctype>
10 #include <stdexcept>
11 #include <gsl/gsl>
12
13 class Cipher {
14 public:
```

Studying the Vigenère Secret-Key Cipher Implementation (Cont.)

- Vigenère Square

- As we mentioned in Chapter 5, simple substitution ciphers like the Caesar cipher are relatively easy to decrypt.
 - ◆ For example, “e” is the most frequently used English letter.
 - ◆ So, you could study English ciphertext and assume that the most frequently appearing character probably is an “e.”
- The Vigenère secret-key cipher **uses letters from the plaintext and a secret key** to locate replacement characters in 26 separate Caesar ciphers—one for each letter of the alphabet.

Studying the Vigenère Secret-Key Cipher Implementation (Cont.)

- Vigenère Square

- These 26 ciphers form a 26-by-26 two-dimensional array called the Vigenère square:

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
A	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
B	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A
C	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B
D	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C
E	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D
F	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E
G	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F
H	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G
I	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H
J	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I
K	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J
L	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K
M	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L
N	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M
O	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N
P	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
Q	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
R	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q
S	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
T	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S
U	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
V	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U
W	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V
X	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W
Y	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X
Z	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y

Studying the Vigenère Secret-Key Cipher Implementation (Cont.)

- Vigenère Square

- You look up substitutions using the bold letters at the left and top that label the rows and columns.
- Our Cipher class's private data, defined at the bottom of the class definition, consists of
 - ◆ a static constant `m_size` (declared at line 153), representing the number of rows and columns in this Vigenère square and
 - ◆ a two-dimensional `std::array` `m_square` (declared at line 156), representing the Vigenère square.

Studying the Vigenère Secret-Key Cipher Implementation (Cont.)

- Cipher Constructor

- The Cipher constructor (lines 16–39) initializes `m_square` with the 26 Caesar ciphers.

```
15 // constructor to initialize the Vigenère square
16 Cipher() {
17     // array to store the 26 characters A-Z that will be
18     // used to initialize each row of the Vigenère square
19     std::array<char, m_size> alphabet{};
20
21     // fill alphabet with A-Z
22     for (size_t i{0}; i < m_size; ++i) {
23         // convert 'A' + i to a char and place in alphabet
24         alphabet.at(i) = gsl::narrow_cast<char>('A' + i);
25     }
26
27     // copy alphabet into row 0 of the Vigenère square
28     m_square.at(0) = alphabet;
29
30     // for each remaining row of the Vigenère square, move alphabet's
31     // first letter to the end then copy alphabet into the row
32     for (int row{1}; row < m_size; ++row) {
33         // rotate alphabet, moving its first letter to the end
34         std::ranges::rotate(alphabet, std::begin(alphabet) + 1);
35
36         // copy alphabet into current row of the Vigenère square
37         m_square.at(row) = alphabet;
38     }
39 }
40
```

Studying the Vigenère Secret-Key Cipher Implementation (Cont.)

- Cipher Constructor

- Line 19 defines the char array alphabet that will initially store the letters A – Z.
- Lines 22–25 fill alphabet with A – Z. Recall from Chapter 8 that 'A' has the value 65 in the underlying character set. Adding the values 0 – 25 to 'A' produces the integer values of 'A' through 'Z'. Line 24 uses `gsl::narrow_cast` to convert the int value to a char and place it in alphabet.

```
15 // constructor to initialize the Vigenère square
16 Cipher() {
17     // array to store the 26 characters A-Z that will be
18     // used to initialize each row of the Vigenère square
19     std::array<char, m_size> alphabet{};
20
21     // fill alphabet with A-Z
22     for (size_t i{0}; i < m_size; ++i) {
23         // convert 'A' + i to a char and place in alphabet
24         alphabet.at(i) = gsl::narrow_cast<char>('A' + i);
25     }
```

Studying the Vigenère Secret-Key Cipher Implementation (Cont.)

- Cipher Constructor

- Line 28 copies alphabet's elements into row 0 of m_square.
- For each of the Vigenère square's remaining rows, lines 32–38 move alphabet's first letter to the end then copy alphabet into the current row.

```
27 // copy alphabet into row 0 of the Vigenère square
28 m_square.at(0) = alphabet;
29
30 // for each remaining row of the Vigenère square, move alphabet's
31 // first letter to the end then copy alphabet into the row
32 for (int row{1}; row < m_size; ++row) {
33     // rotate alphabet, moving its first letter to the end
34     std::ranges::rotate(alphabet, std::begin(alphabet) + 1);
35
36     // copy alphabet into current row of the Vigenère square
37     m_square.at(row) = alphabet;
38 }
39
40
```

Studying the Vigenère Secret-Key Cipher Implementation (Cont.)

- Cipher Constructor

- Line 34 uses the C++20 ranges algorithm `rotate` to move alphabet's first letter to the end and the rest of the its elements toward the beginning, thus “rotating” the elements.
 - ◆ The first argument (`alphabet`) is the range to rotate and the second indicates the first of the range's remaining elements that should rotate toward the beginning.
 - ◆ The expression `std::begin(alphabet) + 1` represents the element one position after the beginning of `alphabet`—that is, its second element.
 - ◆ Elements positioned before the second argument move to the end of the range.

33
34

```
// rotate alphabet, moving its first letter to the end  
std::ranges::rotate(alphabet, std::begin(alphabet) + 1);
```

Studying the Vigenère Secret-Key Cipher Implementation (Cont.)

- Cipher Constructor

- So, if alphabet initially contains A–Z,
 - ◆ after the first rotate call, it will contain B–Z followed by A,
 - ◆ after the second call, it will contain C–Z followed by A–B,
 - ◆ after the second call, it will contain D–Z followed by A–C, etc.
- Recall from Chapter 8 that a benefit of the `std::ranges` algorithms is that you do not need to specify the beginning and end of the range to iterate over—the `std::ranges` algorithms handle this for you, simplifying your code.

Studying the Vigenère Secret-Key Cipher Implementation (Cont.)

- Secret-Key Requirements

- For the Vigenère cipher described here, the secret key must contain only letters.
 - ◆ Like passwords, the secret key should not be easy to guess.
- To create the ciphertext shown in the example, we used as our secret key the following 11 randomly selected characters: XMWUJBVYHXZ
- Your key can have as many characters as you like.
 - ◆ The person decrypting the ciphertext must know the secret key used to create the ciphertext.
- Presumably, you'd provide that in advance—possibly in a face-to-face meeting.
 - ◆ The secret key must be carefully guarded.

Studying the Vigenère Secret-Key Cipher Implementation (Cont.)

- The Vigenère Cipher Encryption Algorithm
 - To see how the Vigenère cipher works, let's use the key "XMWUJBVYHXZ" and encrypt the plaintext string: Welcome to encryption
 - Our encryption and decryption implementations preserve the plaintext's original case.
 - We chose to pass non-letters in the plaintext—like spaces, digits and punctuation—through to the ciphertext and vice versa.

Studying the Vigenère Secret-Key Cipher Implementation (Cont.)

- The Vigenère Cipher Encryption Algorithm

- First, we repeat the secret key until the length matches the plaintext:

Plaintext:

Repeating key text:

W	e	l	c	o	m	e		t	o		e	n	c	r	y	p	t	i	o	n
X	M	W	U	J	B	V		Y	H		X	Z	X	M	W	U	J	B	V	Y

- In this diagram, we highlighted in light blue the secret key, then in darker blue the secret key's eight repeated letters.
- We begin the encryption by using the first letter in the repeating key text ('X') to select a row in the Vigenère square and using the first letter in the plaintext ('W') to select a column.

Studying the Vigenère Secret-Key Cipher Implementation (Cont.)

- The Vigenère Cipher Encryption Algorithm
 - The intersection of that row and column (highlighted below) contains the letter to substitute in the ciphertext for 'W'—in this case, 'T':

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
A	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
B	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A
C	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B
D	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C
E	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D
F	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E
G	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F
H	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G
I	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H
J	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I
K	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J
L	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K
M	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L
N	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M
O	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N
P	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
Q	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
R	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q
S	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
T	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S
U	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
V	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U
W	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V
X	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W
Y	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X
Z	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y

Studying the Vigenère Secret-Key Cipher Implementation (Cont.)

- The Vigenère Cipher Encryption Algorithm
 - This process continues for each pair of letters from the secret key and the plaintext:

Plaintext:

Repeating key text:

Ciphertext:

W	e	l	c	o	m	e		t	o		e	n	c	r	y	p	t	i	o	n
X	M	W	U	J	B	V		Y	H		X	Z	X	M	W	U	J	B	V	Y
T	q	h	w	x	n	z		r	v		b	m	z	d	u	j	c	j	j	l

Studying the Vigenère Secret-Key Cipher Implementation (Cont.)

- Member Function encrypt
 - Function encrypt (lines 43–92) receives the plaintext to encrypt and the secret key and returns encrypted text of string.

```
41 // encrypt receives the plaintext and secret key, applies
42 // the Vigenère cipher returns the encrypted ciphertext
43 std::string encrypt(
44     std::string_view plaintext, std::string_view secret) {
45
46     checkKey(secret); // ensure secret key contains only letters
47
48     std::string ciphertext{}; // stores encrypted text
49     size_t keyIndex{0}; // current letter index in secret key
50
51     // iterate through each character in plaintext
52     for (size_t i{0}; i < plaintext.size(); ++i) {
53         // determine whether character at i is lowercase so we can
54         // place a corresponding lowercase letter in the ciphertext
55         const bool lower{std::islower(plaintext.at(i)) ? true : false};
56
57         // convert currentChar to uppercase; for uppercase
58         // letters and non-letters the character remains the same
59         const char currentChar{gsl::narrow_cast<const char>(
60             std::toupper(plaintext.at(i)))};
```

Studying the Vigenère Secret-Key Cipher Implementation (Cont.)

- Member Function encrypt

- Line 46 calls utility function checkKey (defined lines 160–170) to ensure that the secret key contains only letters.
- Line 48 defines the string ciphertext, which will store the encrypted text.
- Line 49 defines the size_t keyIndex, which will keep track of the index of the current character in the secret key. That character is used to select a Vigenère square row.

```
41 // encrypt receives the plaintext and secret key, applies
42 // the Vigenère cipher returns the encrypted ciphertext
43 std::string encrypt(
44     std::string_view plaintext, std::string_view secret) {
45
46     checkKey(secret); // ensure secret key contains only letters
47
48     std::string ciphertext{}; // stores encrypted text
49     size_t keyIndex{0}; // current letter index in secret key
```

Studying the Vigenère Secret-Key Cipher Implementation (Cont.)

- Member Function encrypt

- For each character in plaintext, lines 52–89 either encrypt the letter and add it to ciphertext or, if the character is not a letter, simply pass it through to the ciphertext.
- Line 55 determines whether the original character was lowercase, so we can maintain that case as we create the ciphertext.

```
51 // iterate through each character in plaintext
52 for (size_t i{0}; i < plaintext.size(); ++i) {
53     // determine whether character at i is lowercase so we can
54     // place a corresponding lowercase letter in the ciphertext
55     const bool lower{std::islower(plaintext.at(i)) ? true : false};
56
57     // convert currentChar to uppercase; for uppercase
58     // letters and non-letters the character remains the same
59     const char currentChar{gsl::narrow_cast<const char>(
60         std::toupper(plaintext.at(i)))};
```

Studying the Vigenère Secret-Key Cipher Implementation (Cont.)

- Member Function encrypt

- Lines 59–60 convert the current character to uppercase.
 - ◆ If it is already uppercase or is not a letter, `std::toupper` returns the original character.
 - ◆ Function `std::toupper`'s result is the int representation of the character, so we convert it to a `const char` to place it in `currentChar`.

```
51 // iterate through each character in plaintext
52 for (size_t i{0}; i < plaintext.size(); ++i) {
53     // determine whether character at i is lowercase so we can
54     // place a corresponding lowercase letter in the ciphertext
55     const bool lower{std::islower(plaintext.at(i)) ? true : false};
56
57     // convert currentChar to uppercase; for uppercase
58     // letters and non-letters the character remains the same
59     const char currentChar{gsl::narrow_cast<const char>(
60         std::toupper(plaintext.at(i)))};
```

Studying the Vigenère Secret-Key Cipher Implementation (Cont.)

- Member Function encrypt

- If currentChar is a letter (line 64), then lines 68–84 encrypt it.
- Line 68 converts the secret's letter at keyIndex to uppercase, then subtracts 'A' to produce a value from 0–26 representing the Vigenère square row to select.
- Line 72 increments the keyIndex to the next position in secret, setting it back to 0 if we've reached the end of secret.

```
62 // if the current character is a letter, encrypt it
63 // and add it to cipherText in its original case
64 if ('A' <= currentChar && currentChar <= 'Z') {
65     // to get the row index in the Vigenère square, select the
66     // character at keyIndex in the secret key, convert it to
67     // uppercase and subtract 'A' from it
68     const int row{std::toupper(secret.at(keyIndex)) - 'A'};
69
70     // increment the keyIndex, ensuring that it is
71     // reset to 0 if keyIndex reaches secret.size()
72     keyIndex = (keyIndex + 1) % secret.size();
```

Studying the Vigenère Secret-Key Cipher Implementation (Cont.)

- Member Function encrypt

- Line 76 calculates the column index by subtracting 'A' from currentChar to produce a value from 0–26 representing the Vigenère square column to select.
- Line 79 performs the Vigenère cipher encryption algorithm for currentChar and stores its ciphertext equivalent in substituteChar.

```
74 // to get the column index in the Vigenère square,  
75 // subtract 'A' from the currentChar  
76 const int column{currentChar - 'A'};  
77  
78 // select the substitute character from the Vigenère square  
79 const char substituteChar{m_square.at(row).at(column)};
```

Studying the Vigenère Secret-Key Cipher Implementation (Cont.)

- Member Function encrypt
 - Lines 83–84 then append substituteChar to ciphertext using currentChar's original case.
 - If currentChar was not a letter, line 87 simply appends it to

```
81         // add substituteChar to the ciphertext, ensuring that
82         // it's lowercase if the plaintext character was lowercase
83         ciphertext +=
84             (lower ? std::tolower(substituteChar) : substituteChar);
85     }
86     else {
87         ciphertext += currentChar; // add non-letter to ciphertext
88     }
89 }
90
91 return ciphertext; // return the encrypted text
92 }
```

Studying the Vigenère Secret-Key Cipher Implementation (Cont.)

- Decrypting with the Vigenère Cipher
 - The decryption process returns the ciphertext to the original plaintext.
 - It's similar to what we described above and requires the same secret key used to encrypt the text.
 - Like the encryption algorithm, the decryption algorithm cycles through the secret key's letters.

Studying the Vigenère Secret-Key Cipher Implementation (Cont.)

- Decrypting with the Vigenère Cipher

- So, again, we repeat the secret key until the length matches the ciphertext:

Ciphertext:

Repeating key text:

T	q	h	w	x	n	z		r	v		b	m	z	d	u	j	c	j	j	l
X	M	W	U	J	B	V		Y	H		X	Z	X	M	W	U	J	B	V	Y

- We begin the decryption by using the first letter in the repeating key text ('X') to select a row in the Vigenère square.
- Next, we locate within that row the first letter in the ciphertext ('T').

Studying the Vigenère Secret-Key Cipher Implementation (Cont.)

- Decrypting with the Vigenère Cipher
 - Finally, we replace the ciphertext letter with the plaintext letter at the top of that column ('W'), as highlighted in the Vigenère square below:

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
A	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
B	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A
C	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B
D	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C
E	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D
F	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E
G	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F
H	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G
I	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H
J	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I
K	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J
L	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K
M	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L
N	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M
O	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N
P	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
Q	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
R	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q
S	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
T	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S
U	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
V	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U
W	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V
X	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W
Y	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X
Z	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y

Studying the Vigenère Secret-Key Cipher Implementation (Cont.)

- Decrypting with the Vigenère Cipher
 - This process continues for each pair of letters from the secret key and the ciphertext:

Ciphertext:

Repeating key text:

Plaintext:

T	q	h	w	x	n	z		r	v		b	m	z	d	u	j	c	j	j	l
X	M	W	U	J	B	V		Y	H		X	Z	X	M	W	U	J	B	V	Y
W	e	l	c	o	m	e		t	o		e	n	c	r	y	p	t	i	o	n

Studying the Vigenère Secret-Key Cipher Implementation (Cont.)

- Member Function decrypt

- Function decrypt (lines 96–150) receives the ciphertext to decrypt and the secret key and returns a string containing the original plaintext.
 - ◆ Line 99 calls utility function checkKey (defined lines 160–170) to ensure that the secret key contains only letters.
 - ◆ Line 101 defines the string plaintext, which will store the unencrypted text.
 - ◆ Line 102 defines the size_t keyIndex, which will keep track of the current character in the secret key.
 - That character is used to select a Vigenère square row.

```
94 // decrypt receives the ciphertext and secret key, reverses
95 // Vigenère cipher process and returns the unencrypted plaintext
96 std::string decrypt(
97     std::string_view ciphertext, std::string_view secret) {
98
99     checkKey(secret); // ensure secret key contains only letters
100
101     std::string plaintext{}; // stores unencrypted text
102     size_t keyIndex{0}; // current letter index in secret key
103
```

Studying the Vigenère Secret-Key Cipher Implementation (Cont.)

- Member Function decrypt

- For each character in plaintext, lines 104–147 either decrypt the letter and add it to plaintext or, if the character is not a letter, simply add it to plaintext.
 - ◆ Line 107 determines whether the original character was lowercase, so we can maintain that case as we create the plaintext.
 - ◆ Lines 111–112 convert the current character to uppercase.
 - If it is already uppercase or is not a letter, `std::toupper` returns the original character.
 - We convert `std::toupper`'s result (an int) to a const char and place it in `currentChar`.

```
104 for (size_t i{0}; i < ciphertext.size(); ++i) {
105     // determine whether character at i is lowercase so we can
106     // place a corresponding lowercase letter in plainText
107     const bool lower{std::islower(ciphertext.at(i)) ? true : false};
108
109     // convert currentChar to uppercase; for uppercase
110     // letters and non-letters the character remains the same
111     const char currentChar{gsl::narrow_cast<const char>(
112         std::toupper(ciphertext.at(i)))};
```

Studying the Vigenère Secret-Key Cipher Implementation (Cont.)

- Member Function decrypt

- ◆ If currentChar is a letter (line 115), then lines 119–142 decrypt it.
- ◆ Line 119 converts the secret's letter at keyIndex to uppercase, then subtracts 'A' to produce a value from 0–26 representing the Vigenère square row to select.
- ◆ Line 123 increments the keyIndex to the next position in secret, setting it back to 0 if we've reached the end of secret.

```
114 // if current is a letter decrypt it
115 if ('A' <= currentChar && currentChar <= 'Z') {
116     // to get the row index in the Vigenère square, select the
117     // character at keyIndex in the secret key, convert it to
118     // uppercase and subtract 'A' from it
119     const int row{std::toupper(secret.at(keyIndex)) - 'A'};
120
121     // increment the keyIndex, ensuring that it is
122     // reset to 0 if keyIndex reaches secret.size()
123     keyIndex = (keyIndex + 1) % secret.size();
```

Studying the Vigenère Secret-Key Cipher Implementation (Cont.)

- Member Function decrypt

- ◆ Lines 126–138 perform the Vigenère cipher decryption algorithm for one character.
 - We determine the Vigenère square column index by iterating through the current row to locate the index (i) of the character that matches currentChar.
 - Then, lines 137–138 add that column to 'A' to determine originalChar.

```
125 // column in the Vigenère square
126 int column{-1};
127
128 // find currentChar's column in Vigenère square's current row
129 for (int i{0}; i < m_square.at(row).size(); ++i) {
130     if (m_square.at(row).at(i) == currentChar) {
131         column = i;
132         break;
133     }
134 }
135
136 // determine original character
137 const char originalChar{
138     gsl::narrow_cast<const char>('A' + column)};
```

Studying the Vigenère Secret-Key Cipher Implementation (Cont.)

- Member Function decrypt

- ◆ Lines 141–142 then append originalChar to plaintext using currentChar's original case.
- ◆ If currentChar was not a letter, line 145 simply appends it to plaintext.

```
140         // add originalChar to plaintext in the correct case
141         plaintext +=
142             (lower ? std::tolower(originalChar) : originalChar);
143     }
144     else {
145         plaintext += currentChar; // add non-letter to plaintext
146     }
147 }
148
149 return plaintext; // return the unencrypted text
150 }
```

Studying the Vigenère Secret-Key Cipher Implementation (Cont.)

- private Data and Utility Function checkKey
 - Line 153 defines the static constant m_size, representing the number of rows and columns in the Vigenère square.
 - Line 156 defines the two-dimensional array of chars m_square, which the constructor (lines 16–39) populates with the Vigenère square.

```
151 private:  
152     // number of rows and columns in the Vigenère square  
153     static constexpr size_t m_size{26};  
154  
155     // 26-by-26 array of characters to store the Vigenère square  
156     std::array<std::array<char, m_size>, m_size> m_square;
```

Studying the Vigenère Secret-Key Cipher Implementation (Cont.)

- private Data and Utility Function checkKey
 - The static utility function checkKey (lines 160–170) receives a secret-key string and ensures that it contains only letters; otherwise, the function throws an `invalid_argument` exception because the key cannot be used with our Vigenère cipher implementation.

```
158 // utility function checks that secret key contains only letters;
159 // throws an invalid_argument exception if key contains non-letters
160 static void checkKey(std::string_view secret) {
161     for (size_t i{0}; i < secret.size(); ++i) {
162         // if the uppercase version of the character at index i
163         // is not a letter throw an invalid_argument exception
164         if (std::toupper(secret.at(i)) < 'A' ||
165             std::toupper(secret.at(i)) > 'Z') {
166             throw std::invalid_argument(
167                 "key must contain only letters A-Z or a-z");
168         }
169     }
170 }
171 ;
```

Studying the Vigenère Secret-Key Cipher Implementation (Cont.)

- Weakness in Secret-Key Cryptography: A Look to Public-Key Cryptography
 - Secret-key encryption and decryption have a weakness—the ciphertext is only as secure as the secret key.
 - ◆ The ciphertext can be decrypted by anyone who discovers or steals the secret key.
 - Public-key cryptography performs encryption with a public key known to every sender who may want to send a secret message to a particular receiver.
 - ◆ The public key can be used to encrypt messages but not decrypt them.
 - ◆ The messages can be decrypted only with a paired private key known only to the receiver, so it's much more secure than the secret key in secret-key cryptography.
 - ◆ One of the most popular public-key cryptography schemes is RSA.

Studying the Vigenère Secret-Key Cipher Implementation.

- A Note about Cryptography and Computing Power
 - Ideally, Ciphertext should be impossible to “break”—that is, it should not be possible to determine the plaintext from the ciphertext without the decryption key.
 - ◆ For various reasons, that goal is impractical.
 - So, designers of cryptography schemes settle for making them extraordinarily difficult to break.
 - One problem with today’s increasingly powerful computers is that they’re making it possible to break most encryption schemes in use over the last few decades.
 - Cryptography is at the root of cryptocurrencies such as Bitcoin.
 - The phenomenally powerful computers that quantum computing will make possible are putting cryptography schemes and cryptocurrencies at risk.
 - ◆ The cryptocurrency community is working on these challenges.

Objects Natural Case Study Exercise: Serialization with JSON

- More and more computing today is done “in the cloud”—that is, distributed across the Internet. Many applications you use daily communicate over the Internet with cloud-based services that use massive clusters of computing resources (computers, processors, memory, disk drives, databases, etc.).
- A service that provides access to itself over the Internet is known as a web service.
- Applications typically communicate with web services by sending and receiving JSON objects.
- JSON (JavaScript Object Notation) is a text-based, human-and-computer-readable data-interchange format that represents objects as collections of name–value pairs.
 - JSON has become the preferred data format for transmitting objects across platforms. In this case study exercise, we’ll explore using JSON in C++.

Objects Natural Case Study Exercise: Serialization with JSON (Cont.)

- JSON Data Format

- Each JSON object contains a comma-separated list of property names and values in curly braces.
 - ◆ For example, the following name–value pairs might represent a client record:
`{"account": 100, "name": "Ito", "balance": 24.98}`
- JSON also supports arrays as comma-separated values in square brackets.
 - ◆ For example, the following represents a JSON array of numbers:
`[100, 200, 300]`

Objects Natural Case Study Exercise: Serialization with JSON (Cont.)

- JSON Data Format

- Values in JSON objects and arrays can be
 - ◆ strings in double quotes (like "Ito"),
 - ◆ numbers (like 100 or 24.98),
 - ◆ JSON Boolean values (represented as true or false),
 - ◆ null (to represent no value),
 - ◆ arrays of any valid JSON value and
 - ◆ other JSON objects.
- JSON arrays may contain elements of the same or different types.

Objects Natural Case Study Exercise: Serialization with JSON (Cont.)

- Serialization

- Converting an object into another format for storage locally or for transmission over the Internet is known as [serialization](#).
- Similarly, reconstructing an object from serialized data is known as deserialization.
- JSON is just one of several serialization formats.
- Another common format is XML (eXtensible Markup Language).

Objects Natural Case Study Exercise: Serialization with JSON (Cont.)

- **Serialization Security**

- Some programming languages have their own serialization mechanisms that use a **language-native format**.
- Deserializing objects using these native serialization formats is a source of various security issues.
- According to the Open Web Application Security Project (OWASP), these native mechanisms “can be repurposed for malicious effect when operating on untrusted data.
- Attacks against deserializers have been found to allow denial-of-service, access control, and remote code execution (RCE) attacks.”
- OWASP also indicates that you can significantly reduce attack risk by avoiding language-native serialization formats in favor of “pure data” formats like JSON or XML.

Objects Natural Case Study Exercise: Serialization with JSON (Cont.)

- cereal Header-Only Serialization Library
 - The cereal header-only library serializes objects to and deserializes objects from JSON (which we'll demonstrate), XML or binary formats.
 - The library supports fundamental types and can handle most standard library types if you include each type's appropriate cereal header.
 - ◆ As you'll see, cereal also supports custom types. The cereal documentation is available at <https://uscilab.github.io/cereal/index.html>
 - We've included the cereal library for your convenience in the libraries folder with the examples.
 - ◆ You must point your IDE or compiler at the library's include folder, as you've done in several earlier Objects-Natural case studies.

Objects Natural Case Study Exercise: Serialization with JSON (Cont.)

- Serializing a vector of Objects Containing public Data
 - Let's begin by serializing objects containing only public data. In the example, we
 - ◆ create a vector of Record objects and display its contents,
 - ◆ use cereal to serialize it to a text file, then
 - ◆ deserialize the file's contents into a vector of Record objects and display the deserialized Records.
 - To perform JSON serialization, include the cereal header `json.hpp` (line 3).
 - To serialize a `std::vector`, include the cereal header `vector.hpp` (line 4).

```
1 // main.cpp
2 // Serializing and deserializing objects with the cereal library.
3 #include <cereal/archives/json.hpp>
4 #include <cereal/types/vector.hpp>
5 #include <format>
6 #include <fstream>
7 #include <iostream>
8 #include <vector>
```

Objects Natural Case Study Exercise: Serialization with JSON (Cont.)

- Aggregate Type Record

- Record is an aggregate type defined as a struct.
- Recall that an aggregate's data must be public, which is the default for a struct definition.

```
10 struct Record {  
11     int account{};  
12     std::string first{};  
13     std::string last{};  
14     double balance{};  
15 };
```

Objects Serializa

```
17 // function template serialize is responsible for serializing and
18 // deserializing Record objects to/from the specified Archive
19 template <typename Archive>
20 void serialize(Archive& archive, Record& record) {
21     archive(cereal::make_nvp("account", record.account),
22            cereal::make_nvp("first", record.first),
23            cereal::make_nvp("last", record.last),
24            cereal::make_nvp("balance", record.balance));
25 }
```

- Function serialize for Record Objects

- The cereal library allows you to designate how to perform serialization several ways.
- If the types you wish to serialize have all public data, you can simply define a function template `serialize` (lines 19–25) that receives an `Archive` as its first parameter and an object of your type as the second.
 - ◆ This function is called both for serializing and deserializing the `Record` objects.
 - ◆ Using a function template enables you to choose among serialization and deserialization using JSON, XML or binary formats by passing an object of the appropriate cereal archive type.
 - ◆ The library provides archive implementations for each case.

Objects Natural Case Study Exercise: Serialization with JSON (Cont.)

- Function serialize for Record Objects

- Each cereal archive type has an overloaded parentheses operator that enables you to use an archive parameter as a function name, as in lines 21–24.
- Depending on whether you're serializing or deserializing a Record, this function will either
 - ◆ output the contents of the Record to a specified stream or
 - ◆ input previously serialized data from a specified stream and create a Record object.

```
17 // function template serialize is responsible for serializing and
18 // deserializing Record objects to/from the specified Archive
19 template <typename Archive>
20 void serialize(Archive& archive, Record& record) {
21     archive(cereal::make_nvp("account", record.account),
22            cereal::make_nvp("first", record.first),
23            cereal::make_nvp("last", record.last),
24            cereal::make_nvp("balance", record.balance));
25 }
```

Objects Natural Case Study Exercise: Serialization with JSON (Cont.)

- Function `serialize` for Record Objects

- Each call to `cereal::make_nvp` (short for “make name–value pair”), like line 21

`cereal::make_nvp("account", record.account)`
is primarily for the serialization step.

- ◆ It makes a name–value pair with the name in the first argument (in this case, "account") and the value in the second argument (in this case, the int value `record.account`).
- ◆ Naming the values is not required but makes the JSON output more readable, as you’ll soon see.

➤ Otherwise, `cereal` uses names like `value0`, `value1`, etc.

```
17 // function template serialize is responsible for serializing and
18 // deserializing Record objects to/from the specified Archive
19 template <typename Archive>
20 void serialize(Archive& archive, Record& record) {
21     archive(cereal::make_nvp("account", record.account),
22            cereal::make_nvp("first", record.first),
23            cereal::make_nvp("last", record.last),
24            cereal::make_nvp("balance", record.balance));
25 }
```

Objects Natural Case Study Exercise: Serialization with JSON (Cont.)

- Function displayRecords

- We provide function displayRecords to show you the contents of our Record objects before and after deserialization.
- The function simply displays the contents of each Record in the vector it receives as an argument.

```
27 // display record at command line
28 void displayRecords(const std::vector<Record>& records) {
29     for (const auto& r : records) {
30         std::cout << std::format("{} {} {} {:.2f}\n",
31             r.account, r.first, r.last, r.balance);
32     }
33 }
```

Objects Natural Case Study Exercise: Serialization with JSON (Cont.)

- Creating Record Objects to Serialize

- Lines 36–39 in main create a vector and initialize it with two Records (lines 37 and 38).
 - ◆ The compiler uses class template argument deduction (CTAD) to determine the vector's element type from the Records in the initializer list. Each Record is initialized with aggregate initialization.
- Line 42 outputs the vector's contents to confirm that the two Records were initialized properly.

```
35 int main() {
36     std::vector records{
37         Record{100, "Bakary", "Zongo", 123.45},
38         Record{200, "Sofia", "Smirnova", 987.65}
39     };
40
41     std::cout << "Records to serialize:\n";
42     displayRecords(records);
```

Records to serialize: 100 Bakary Zongo 123.45 200 Sofia Smirnova 987.65

Objects Natural Case Study Exercise: Serialization with JSON (Cont.)

- Serializing Record Objects with `cereal::JSONOutputArchive`
 - A `cereal::JSONOutputArchive` serializes data in JSON format to a specified stream, such as the standard output stream or a stream representing a file.
 - Line 45 attempts to open the file `records.json` for writing. If it is successful,

```
44 // serialize vector of Records to JSON and store in text file
45 if (std::ofstream output{"records.json"}) {
46     cereal::JSONOutputArchive archive{output};
47     archive(cereal::make_nvp("records", records)); // serialize records
48 }
```

Objects Natural Case Study Exercise: Serialization with JSON (Cont.)

- Serializing Record Objects with `cereal::JSONOutputArchive`
 - line 46 creates a `cereal::JSONOutputArchive` object named `archive` and initializes it with the `ofstream` object `output` so `archive` can write the JSON data into a file.
 - Line 47 uses the `archive` object to output a name–value pair with the name `"records"` and the vector of `Records` as its value.
 - ◆ Part of serializing the vector is serializing each of its elements. So line 47 also results in one call to `serialize` (lines 19–25) for each `Record` object in the vector.

```
44 // serialize vector of Records to JSON and store in text file
45 if (std::ofstream output{"records.json"}) {
46     cereal::JSONOutputArchive archive{output};
47     archive(cereal::make_nvp("records", records)); // serialize records
48 }
```

Objects Natural Case Study

Serialization with JSON (C++)

- Contents of records.json

- After line 47 executes, the file records.json contains the above JSON data:

```
{  
  "records": [  
    {  
      "account": 100,  
      "first": "Bakary",  
      "last": "Zongo",  
      "balance": 123.45  
    },  
    {  
      "account": 200,  
      "first": "Sofia",  
      "last": "Smirnova",  
      "balance": 987.65  
    }  
  ]  
}
```

- ◆ The outer braces represent the entire JSON document.
- ◆ The darker box highlights the document's one name–value pair named "records", which has as its value a JSON array containing the two JSON objects in the lighter boxes.
- ◆ The JSON array represents the vector records that we serialized in line 47.
- ◆ Each of the JSON objects in the array contains one Record's four name–value pairs that were serialized by the serialize function.

```
44 // serialize vector of Records to JSON and store in text file  
45 if (std::ofstream output{"records.json"}) {  
46     cereal::JSONOutputArchive archive{output};  
47     archive(cereal::make_nvp("records", records)); // serialize records  
48 }
```

Objects Natural Case Study Exercise: Serialization with JSON (Cont.)

- Deserializing Record Objects with `cereal::JSONInputArchive`
 - Next, let's deserialize the data and use it to fill a separate vector of Record objects.
 - For cereal to recreate objects in memory, it must have access to each type's default constructor.
 - ◆ It will use that to create an object, then directly access that object's data members to place the data into the object.
 - ◆ Like classes, the compiler provides a public default constructor for structs if you do not define a custom constructor.

```
50 // deserialize JSON from text file into vector of Records
51 if (std::ifstream input{"records.json"}) {
52     cereal::JSONInputArchive archive{input};
53     std::vector<Record> deserializedRecords{};
54     archive(deserializedRecords); // deserialize records
55     std::cout << "\nDeserialized records:\n";
56     displayRecords(deserializedRecords);
57 }
58 }
```

```
Deserialized records:
100 Bakary Zongo 123.45
200 Sofia Smirnova 987.65
```

Objects Natural Case Study Exercise: Serialization with JSON (Cont.)

- Deserializing Record Objects with `cereal::JSONInputArchive`
 - A `cereal::JSONInputArchive` deserializes JSON format from a specified stream.
 - Line 51 attempts to open the file `records.json` for reading.
 - If it is successful, line 52 creates the `cereal::JSONInputArchive` object archive and initializes it with the `ifstream` object input so archive can read JSON data from the `records.json` file.
 - Line 53 creates an empty vector of `Records` into which we'll read the JSON data.

```
50 // deserialize JSON from text file into vector of Records
51 if (std::ifstream input{"records.json"}) {
52     cereal::JSONInputArchive archive{input};
53     std::vector<Record> deserializedRecords{};
54     archive(deserializedRecords); // deserialize records
55     std::cout << "\nDeserialized records:\n";
56     displayRecords(deserializedRecords);
57 }
58 }
```

Deserialized records:
100 Bakary Zongo 123.45
200 Sofia Smirnova 987.65

Objects Natural Case Study Exercise: Serialization with JSON (Cont.)

- Deserializing Record Objects with `cereal::JSONInputArchive`
 - Line 54 uses the archive object to deserialize the file's data into the `deserializedRecords` object.
 - ◆ Part of deserializing the vector is deserializing its elements.
 - ◆ This again results in calls to `serialize` (lines 19–25), but because `archive` is a `cereal::JSONInputArchive`, each call to `serialize` reads one `Record`'s JSON data, creates a `Record` object and inserts the data into it.

```
50 // deserialize JSON from text file into vector of Records
51 if (std::ifstream input{"records.json"}) {
52     cereal::JSONInputArchive archive{input};
53     std::vector<Record> deserializedRecords{};
54     archive(deserializedRecords); // deserialize records
55     std::cout << "\nDeserialized records:\n";
56     displayRecords(deserializedRecords);
57 }
58 }
```

```
Deserialized records:
100 Bakary Zongo 123.45
200 Sofia Smirnova 987.65
```

Objects Natural Case Study Exercise: Serialization with JSON (Cont.)

- Serializing a vector of Objects Containing private Data
 - It is also possible to serialize objects containing private data.
 - To do so, you must **declare the serialize function as a friend** of the class, so it can access the class's private data.
 - To demonstrate serializing private data, create a copy of the previous program and replace the aggregate Record definition with the class Record.
 - This Record class should contain private data, a constructor and get member functions that return each data member's value.
 - The constructor's parameters should all have default arguments, allowing cereal to use this as the default constructor when deserializing Record objects.

Objects Natural Case Study Exercise: Serialization with JSON.

- Serializing a vector of Objects Containing private Data
 - Use the following code in class Record to specify that the serialize function is a friend of the class:
template<typename Archive>
friend void serialize(Archive& archive, Record& record);
 - The serialize function now can access class Record's private data members.
 - The function displayRecords should use each Record's get functions to access the data to display.
 - The main function does not require changes and should produce the same results shown previously.